



Domain-Driven Design in software development: A systematic literature review on implementation, challenges, and effectiveness[☆]

Ozan Özkan^{a,*,}, Önder Babur^{a,b,}, Mark van den Brand^{a,}

^a Mathematics and Computer Science, Eindhoven University of Technology, Eindhoven, The Netherlands

^b Information Technology Group, Wageningen University and Research, Wageningen, The Netherlands

ARTICLE INFO

Keywords:

Domain-Driven Design
DDD
Software architecture
Software development
Systematic literature review

ABSTRACT

Context: Domain-Driven Design (DDD) has gained significant attention in software development for its potential to address complex software challenges, particularly in the areas of system refactoring, reimplementa-

Objective: This SLR aims to provide an analysis of existing research on DDD in software development, paint a picture of DDD in solving software problems, identify the challenges encountered during its application and explore the results of these studies.

Method: We systematically selected 36 peer reviewed studies and conducted quantitative and qualitative analyzes to synthesize the findings.

Results: DDD has effectively improved software systems, with its key concepts. The application of DDD in microservices has gained prominence for its ability to facilitate system decomposition. Some studies lacked empirical evaluations, highlighting challenges in onboarding and the need for expertise.

Conclusion: Adopting DDD benefits software development, involving stakeholders such as engineers, architects, managers, and domain experts. More empirical evaluations and open discussions on challenges are needed. Collaboration between academia and industry advances the adoption and transfer of knowledge of DDD in projects.

1. Introduction

In the field of software development, the pursuit of effective design methodologies and architectural approaches has been a crucial endeavor for both researchers and practitioners. Among these approaches, DDD has gained significant attention and adoption, particularly in the context of solving software architecture problems and addressing challenges related to refactoring, reimplementa-

tion and adoption. Using domain knowledge, DDD aims to solve complex business problems effectively. This approach provides guidance on how to improve system quality, maintainability, and scalability by ensuring that software design closely mirrors the business domain (Evans, 2004).

Although DDD presents solutions with its conceptual elegance and potential benefits in managing software architecture challenges (Evans, 2004; Vernon, 2013), it is crucial to evaluate its practical application and assess its impact on software development projects. This Systematic Literature Review (SLR) aims to provide a comprehensive analysis of existing research studies that have used DDD for various purposes in software development, such as improving system design, enhancing maintainability, and facilitating communication between stakeholders. By analyzing and synthesizing empirical studies, we aim to gain insight into the effectiveness of DDD in solving software architecture problems and identify the challenges encountered during the application of DDD from these studies.

DDD emphasizes the alignment of software systems with the underlying business domain, enabling developers to better understand and model complex business requirements. By capturing the language, concepts, and relationships within the domain, DDD helps to identify and address software architecture challenges effectively (Evans, 2004; Vernon, 2013). This approach provides guidance on how to improve system quality, maintainability, and scalability by ensuring that software design closely mirrors the business domain (Evans, 2004).

[☆] Editor: Patricia Lago.

* Corresponding author.

E-mail address: o.ozkan@tue.nl (O. Özkan).

To achieve this objective, a rigorous research methodology was employed that adheres to established guidelines proposed by Kitchenham and Charters (2007) and Wohlin (2014). A group of 36 peer-reviewed studies was systematically selected and quantitative and qualitative analyses were performed based on the data extracted from these studies.

The conclusion drawn from this SLR emphasizes the potential benefits of adopting DDD in software development. The study highlights the importance of involving key stakeholders, including software engineers, architects, project managers, and domain experts, to ensure a successful implementation. In addition, the study identifies the importance of open discussions on challenges and limitations and emphasizes the collaboration between academia and industry to advance the adoption of DDD and the transfer of knowledge in real-world projects.

This study contributes to the growing body of knowledge on DDD and provides valuable insights for practitioners and researchers seeking to apply the principles of DDD effectively in software development. By identifying the strengths and weaknesses of existing research and proposing areas for improvement, this SLR aims to foster the further advancement and adoption of DDD principles in contemporary software development projects.

1.1. Domain-Driven Design essentials

DDD is a conceptual framework and approach to software development in software engineering that seeks harmony between the structure of software code and the complexities of the business domain it serves. The goal of DDD is to bridge the gap between technical implementation and business requirements, resulting in more effective communication between developers and domain experts (Evans, 2004; Vernon, 2013). This concept was pioneered by Eric Evans, whose significant book *Domain-Driven Design: Tackling Complexity in the Heart of Software* (Evans, 2004) has shaped this paradigm. Evans emphasizes the alignment of software design with the intricate nuances of the business domain. He argues that effective software development requires a deep understanding of the complexities of the domain, allowing developers to create software that genuinely resonates with real-world needs (Evans, 2004).

In DDD, the domain refers to the specific area of expertise or the subject matter of the software application. This could be anything from e-Commerce, finance, healthcare, or logistics. The central concept in DDD is the development of a rich, well-defined domain model that accurately represents the business rules, processes, and entities within the domain. This model is built through collaboration between developers and domain experts, often using a shared language called *Ubiquitous Language* that captures both technical and domain-specific terms.

Through its concepts, which are explained in the following sections, DDD introduces principles that help software architects effectively represent the complex aspects of the real world. By embracing these core tenets, software engineers navigate the complex landscape of software development with a compass calibrated to the nuances of the domain, thus achieving a symbiotic fusion of technical precision and domain understanding.

1.1.1. Ubiquitous language

At the core of DDD lies the concept of Ubiquitous Language, a practice that aims to bridge the gap between technical jargon and domain-specific vocabulary (Evans, 2004). Ubiquitous Language fosters a shared vocabulary that harmonizes conversations among developers, domain experts, and business stakeholders. By encapsulating the complexities of the business domain in a common understanding, Ubiquitous Language eliminates ambiguity and streamlines communication. This shared linguistic framework empowers stakeholders to articulate domain intricacies effortlessly and translates directly into the software's code, facilitating a direct and unambiguous translation of domain understanding into the software's design and implementation.

1.1.2. Bounded context

The Bounded Context concept effectively sets the boundaries of distinct zones where the semantics of a domain model are consistently applied. In applications of substantial scale that incorporate a variety of domain models, there is an inherent risk of ambiguity and misalignment. Bounded Contexts serve as strategic divisions, separating different domain models to maintain their integrity. This approach confines each domain to specific contexts, thus preventing semantic conflicts and ensuring a unified representation of the business domain within each delimited area (Evans, 2004). It can serve as an abstraction of a (sub-)system or team, aligning boundaries with software components, organizational structures, and responsibilities.

1.2. Context mapping

In DDD, Context Mapping is a strategic practice used to understand and manage the relationships between different bounded contexts in complex software systems (Evans, 2004). A Bounded Context defines the boundary within which a particular domain model is defined and applicable. However, in real-world systems, multiple teams often work on different subdomains or systems, each with its own Bounded Context. These contexts must interact and integrate, which introduces the risk of model contamination or ambiguity across teams.

Context Mapping provides a visual and conceptual technique to explicitly define how bounded contexts relate to each other and how integration should be handled. It helps development teams determine appropriate integration patterns (e.g., Shared Kernel, Customer/Supplier, Conformist, or Anti-Corruption Layer) depending on the nature of team collaboration, legacy constraints, or organizational structure. These relationships are represented using a Context Map, which becomes an essential communication artifact in large systems involving multiple teams and domains.

Evans emphasizes that strategic design decisions are essential when teams develop models in parallel, and that context mapping serves as a bridge to align models and avoid conflicts (Evans, 2004). In practice, context maps also guide technical implementation, by informing decisions about API design, data ownership, and service integration. As such, context mapping is a vital part of ensuring semantic consistency and architectural integrity across distributed systems.

1.2.1. Aggregates

The concept of Aggregates encapsulates Entities, Value Objects, and Domain Logic through methods in entities and the use of domain services. Aggregates are the main points for accessing and changing data, ensuring data integrity within business transactions (Evans, 2004). By containing data within defined boundaries, Aggregates optimize data management and system scalability while also enforcing logical boundaries that prevent uncontrolled data manipulation.

1.2.2. Entities and value objects

Entities and Value Objects differentiate within the DDD framework. Entities embody tangible elements within the domain, each possessing a unique identity. These entities change over time, capturing the mutable aspects of the business reality (Evans, 2004). In contrast, Value Objects represent conceptual attributes that lack identity and remain immutable. By distinguishing between these two constructs, DDD provides a complete representation of the domain, accurately modeling tangible entities and abstract attributes (Evans, 2004).

1.2.3. Domain services

Domain Services encapsulate domain logic that transcends individual entities and value objects (Evans, 2004). These services orchestrate complex operations and interactions within the domain. By segregating multifaceted logic, domain services promote modular, reusable, and concise code. This approach mirrors the coherent coordination of real-world processes, enhancing the software's alignment with the domain's intricacies.

1.2.4. Domain events

Domain Events is a concept used to model and communicate changes or significant occurrences within a domain [Vernon \(2013\)](#). They play a role in capturing and representing the behavior and state changes within an application's core domain. A Domain Event is a lightweight object that represents something important that has happened in the domain. These events are usually named in the past tense to indicate that they have already occurred. They help decouple different parts of a system by allowing different components or aggregates to communicate without needing to know the specifics of each other. This promotes a more modular and maintainable architecture.

1.2.5. Anti-Corruption Layer (ACL)

The concept of an ACL protects the domain model from external influences. This layer mediates interactions between different models or systems, preserving the domain's integrity. By translating data between disparate representations, the ACL ensures a consistent domain model, safeguarding it from external distortions [\(Evans, 2004\)](#).

1.2.6. Core domain

Core Domain is a central concept of strategic design in DDD, which refers to the part of the software system that delivers the most value and differentiates the business. The goal is to focus development effort on the Core Domain to gain competitive advantage, often assigning the most skilled developers to it [\(Evans, 2004\)](#).

1.2.7. Shared kernel

Shared Kernel refers to a pattern in which two Bounded Contexts share a common subset of the domain model. This shared part must be carefully managed to avoid conflicts and preserve the integrity of the model, often requiring close collaboration and coordination between teams [\(Evans, 2004\)](#).

The ACL shares similarities with several well-known design patterns. For instance, the *Remote Proxy* pattern [\(Gamma et al., 2009\)](#) provides a local representative for an object that resides in a different address space, mediating interactions between local and remote systems. The *Wrapper* pattern [\(Buschmann, 1996\)](#) encapsulates an object to provide a different interface, much like the ACL encapsulates external systems to present a consistent interface to the domain model. The *Adapter* pattern [\(Gamma et al., 2009\)](#) converts the interface of a class into another interface that a client expects, similar to how the ACL adapts external data representations to match the internal domain model.

The fundamental contribution of DDD lies in creating software systems that closely mirror the business's cognitive schema. By converging domain experts, business stakeholders, and software developers, DDD facilitates synergistic collaboration to deliver software solutions that optimally cater to business needs. This collaborative effort fosters the combination of domain insights, augments the comprehension of problems and solutions across the entirety of the development team, and supports the fostering of inter-team relationships.

1.2.8. Strategic and tactical DDD

Over time, the DDD community has categorized DDD patterns into two broad groups: *Strategic* and *Tactical*. Evans also has a similar categorization in his book [\(Evans, 2004\)](#) as he calls it *Strategic Design*, but the community calls it as *Tactical Design* as of today. It has become a widely adopted framing, especially in practitioner literature such as Vernon's work [\(Vernon, 2013\)](#). Strategic DDD focuses on aligning the software model with the broader business strategy and organization, with key concepts including Bounded Context, Core Domain, Context Maps, and Anti-Corruption Layer. In contrast, Tactical DDD includes concrete modeling techniques used within a Bounded Context, such as Entities, Value Objects, Aggregates, Domain Services, and Domain Events. Throughout this study, we use this distinction as a lens to interpret the usage patterns of DDD concepts in the literature.

1.2.9. Event storming

In addition to modeling patterns, DDD also includes collaborative modeling techniques that help bridge the gap between business and technical stakeholders. One widely adopted practice is Event Storming, a workshop-based approach introduced by Alberto Brandolini [\(Brandolini, 2013\)](#), where domain experts and developers collaboratively explore business processes by identifying and organizing domain events, commands, aggregates, and actors. This method is particularly effective for discovering Bounded Contexts and aligning the domain model with real-world business operations.

1.2.10. Other DDD patterns and concepts

The DDD patterns and concepts introduced in this section reflect those discussed most frequently in the 36 primary studies reviewed. Although there are additional patterns in the DDD literature, such as Customer/Supplier, Open Host Service, Conformist, and Published Language, these were not consistently represented in the reviewed studies and, therefore, were not included in detail. Our goal in this section is to provide a focused overview of the key patterns that emerged from the literature rather than an exhaustive catalog of all DDD concepts.

2. Related work

To the best of our knowledge, despite the growing interest in DDD as a software development approach, there is a scarcity of SLRs specifically focused on DDD. While the literature landscape on DDD is not completely unexplored, previous studies have primarily touched upon certain aspects remotely related to DDD.

For instance, [Le et al. \(2016\)](#) propose a set of concrete design patterns for Domain Modeling within the context of Object-Oriented DDD, providing valuable insights into the practical application of DDD in specific modeling scenarios. [Singai et al. \(2021a\)](#), on the other hand, examine the interrelation between microservice APIs and DDD, shedding light on how DDD principles can be applied in the context of microservices architecture. However, it is worth noting that some related systematic literature reviews have been conducted in adjacent domains. [De Goey et al. \(2019\)](#), for instance, undertakes a Systematic Literature Review on Design-Driven Innovation, exploring design-driven approaches and methodologies in innovation processes, which, although related to design-driven practices, do not directly address the specific principles and patterns of DDD. Similarly, [Bertoni \(2020\)](#) conducts an SLR on Data-Driven Design in Concept Development, focusing on the role of data-driven approaches in the early stages of concept development but not explicitly addressing the nuances of DDD.

Despite valuable contributions from these studies, none of them provide a comprehensive review of DDD as a whole, addressing its principles, methodologies, and practical applications. While these prior studies offer valuable perspectives related to design-driven or microservice-oriented software development, they do not conduct a structured or comprehensive SLR focused on the conceptual and practical aspects of DDD itself. In contrast, our study takes a DDD-centric perspective, systematically analyzing how DDD principles, patterns, and stakeholders are discussed and applied in peer-reviewed literature. This focus enables us to capture insights specifically tied to DDD adoption, effectiveness, and evaluation, filling a gap not directly addressed by adjacent literature.

3. Research objectives and method

This study employed a Systematic Literature Review methodology to create generalizable empirical results (as opposed to specific and in-depth yet less generalizable research methods such as case studies [\(Stol and Fitzgerald, 2018\)](#)) to (1) consolidate the existing knowledge on the utilization of DDD in the domain of software development, (2) identify gaps in the current research, guiding future studies and helping to

focus on underexplored areas, and (3) for practitioners, a comprehensive review can offer insights into best practices, common challenges, and effective strategies for implementing DDD in real-world projects. The research approach adhered to established guidelines as proposed by Lübkke et al. (2019) and Wohlin (2014).

Additionally, this study delves into further detail regarding the Goal and Research Questions (Section 3.1), Search Strategy and Data Sources (Section 3.2), Exclusion Criteria (Section 3.3), Primary Study Identification and Selection (Section 3.4), and Data Extraction (Section 3.5) in subsequent sections. By examining these aspects in a meticulous manner, this study aims to provide a comprehensive and rigorous analysis of the research topic at hand.

3.1. Goal and research questions

The primary objective of this research is to investigate and synthesize the current state of research on the application of DDD in software development, to understand its benefits, challenges, used patterns, and effectiveness. To achieve this objective, the following research questions (RQs) have been formulated:

RQ1: What is the state of the art concerning DDD in the existing literature, in terms of study types, research venues, software systems, and the distribution between academic and industrial settings?

RQ2: What categories of software systems have demonstrated benefits from the implementation of DDD?

RQ3: Which specific software development concerns have been addressed through the application of DDD in previous studies?

RQ4: What are the most common DDD patterns utilized in previous research, such as ACLs, Bounded Contexts, and other related approaches?

RQ5: What are the challenges encountered during the implementation of DDD in software development projects?

RQ6: How has the effectiveness of DDD been assessed and measured in prior studies?

RQ7: Who are the key stakeholders involved and benefited in the implementation of DDD?

3.2. Search strategy and data sources

In line with established protocols and our previous studies (Giray et al., 2023; Ochoa et al., 2025), we meticulously designed our search strategy to identify pertinent studies from reputable academic databases renowned for their extensive coverage of computer science and software engineering literature. The databases utilized in this study were *ACM*, *SpringerLink*, *IEEE*, and *Wiley*, all of which are widely recognized for their authoritative and diverse repositories of articles.

The search query employed in this investigation was thoughtfully crafted to encompass a broad spectrum of articles relevant to our research objectives. The query utilized a combination of keywords and logical operators to target studies explicitly discussing DDD in conjunction with software development activities. We used “Domain-Driven Design” because we explicitly wanted to capture studies that employed DDD. When using only “Domain-Driven Design”, we encountered a significant number of false positives, such as studies from medicine. Therefore, to ensure the relevance of the captured studies, we included implementation keywords to specifically address studies that employed DDD in practice. The search query was formulated as follows:

"Domain Driven Design" AND (refactor* OR improv* OR optimiz* OR restructur* OR migrat* OR evolution OR cleanup OR reengineering)

We strategically used asterisks (*) as wildcards to capture variations of the specified keywords and to ensure inclusivity in the search results.

Table 1

List of exclusion and inclusion criteria.

Exclusion criteria	
EC1	Duplicate papers from multiple sources.
EC2	Papers without full text available.
EC3	Papers not written in English.
EC4	Books, book chapters, editorials, issue introductions.
EC5	Papers are not primary studies.
EC6	Papers do not explicitly introduce a method, technique, tool and/or evaluation of adaptation or extension.
Inclusion Criterion	
Papers do not meet any exclusion criteria.	

3.3. Exclusion criteria

The establishment of selection criteria was undertaken with the aim of mitigating bias and minimizing subjectivity within the study. The inclusion and exclusion criteria were formulated in accordance with the research questions (RQs) as delineated by Kuhrmann et al. (2017), thereby ensuring that the criteria align closely with the intended objectives of the investigation. Following our criteria, (1) we meticulously identified and removed duplicate studies that appeared in more than one of the selected databases to prevent the inclusion of redundant information. To maintain the comprehensiveness of our analysis, (2) we excluded studies that lacked full-text availability, as the absence of complete content may hinder a thorough evaluation of the study’s findings and methodology. Given the language limitations of our review team, (3) we restricted our analysis to studies written in English to ensure accurate and consistent interpretation and analysis. In the interest of focusing on primary research articles, (4) we excluded books, book chapters, editorials, and issue introductions from our analysis, as they may not present original empirical studies or detailed investigations. (5) We excluded studies that were not primary research studies (e.g., review articles, meta-analyses, opinion pieces) to maintain a clear focus on empirical research directly relevant to our research questions. To ensure the inclusion of studies closely related to DDD and its practical applications, (6) we excluded studies that did not explicitly present or discuss the introduction of a method, technique, tool, or evaluation pertaining to DDD or its adaptation and extension. The list of the exclusion criteria can be seen in Table 1.

3.4. Primary study identification and selection

3.4.1. Conducting manual database search

In the first stage, we performed a meticulous and systematic manual search of the databases mentioned in Section 2.2. The search was carried out in March 2023 and yielded a total of 707 studies, which were 153 studies from ACM, 19 studies from IEEE Xplore, 510 studies from SpringerLink, and 25 studies from Wiley, as also shown in Table 4. To ensure the exclusion of duplicate results, we conducted conflict resolution, resulting in the identification of one duplicate paper. After eliminating duplicates, we advanced 706 unique studies to the next stage of the selection process.

3.4.2. Applying exclusion criteria

In the second stage, we applied our predefined exclusion criteria to the 706 studies. The screening process involved the assessment of each paper based on its title, abstract, and keywords. In situations where a thorough evaluation was not possible based on these elements, we briefly reviewed the introduction and conclusion of the paper, following the guidelines suggested by Wohlin (2014). To ensure a rigorous assessment, we adopted a multi-assessor strategy, where 2 authors independently evaluated the studies without knowing each other’s assessments. Due to the high number of studies, we performed the assessment in iterations until we achieved at least 0.80 Cohen’s Kappa

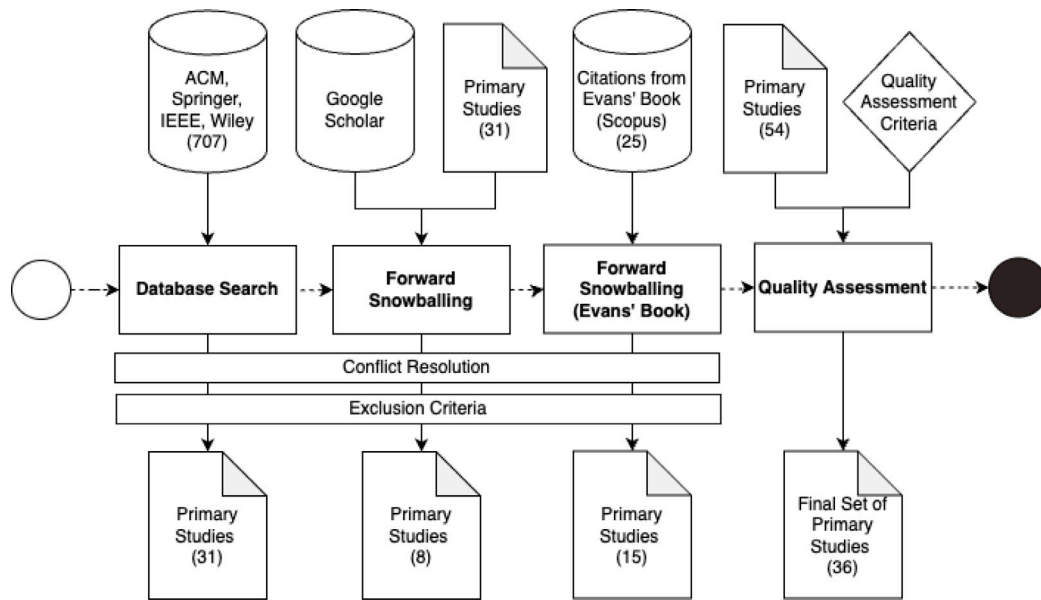


Fig. 1. Study selection progressed from a database search to the quality assessment. The final set of primary studies advanced to the data extraction stage. The count of studies in the evaluation steps is displayed in parentheses.

Table 2
Number of studies excluded per criterion during the screening process.

Count of studies excluded per criterion	
EC1	11 studies excluded as duplicates across databases.
EC2	15 studies excluded due to lack of full-text access.
EC3	0 studies excluded for being in non-English languages.
EC4	461 studies excluded as books, editorials, or chapters.
EC5	53 studies excluded as they were not primary studies.
EC6	147 studies excluded for not introducing or evaluating DDD methods, tools, or adaptations.
Total	687 studies excluded across all criteria.

coefficient (Cohen, 1960), which indicates almost perfect agreement between the assessors (Cohen, 1960). Once the desired Cohen's Kappa was reached, the first author continued to assess the remaining studies to determine their inclusion or exclusion, following the approach done in previous studies (Torres et al., 2021; Wang et al., 2023). In each iteration, we randomly selected 30 studies for assessment. After each iteration, we conducted conflict resolution sessions to discuss the rationale of our assessments. We eventually achieved 1.0 Cohen's Kappa coefficient in the second iteration. The remaining studies were evaluated by the first author, resulting in 31 studies marked for inclusion and progression to the next stage. The distribution of these included studies across the search databases is presented in Table 4.

Table 2 provides a detailed breakdown of the number of studies excluded under each exclusion criterion. This complements the overall metrics shown in Table 4, providing transparency on how rigorously the exclusion process was applied.

3.4.3. Conducting forward snowballing

In the third stage, we conducted forward snowballing to make our study population wider and ensure relevancy. We conducted the forward snowballing in two approaches. We conducted a forward snowballing process for studies that cited Evans' book (Evans, 2004) since Evans holds significant prominence in the field of DDD due to his influential contributions. His book, which serves as a reference, has gained substantial recognition and is widely regarded as a crucial resource among practitioners in the software development community (Avram and Marinescu, 2006; Vernon, 2013, 2016; Nilsson, 2006). To conduct this snowballing, we utilized Scopus as a database because of its

centralized indexing of our target databases mentioned in Section 3.2 and to eliminate results from grey literature. The screening process for Evans' book forward snowballing followed the same strategy as that used for forward snowballing in the primary studies. In the process of analysis, a comparison was made among the database search results, forward snowballing results, and the previously identified studies to identify any potential duplicates. As a result, a total of 10 duplicate studies were identified, and these duplicates were subsequently resolved in favor of the database search results and forward snowballing outcomes. Consequently, an additional 15 studies that did not meet any of the exclusion criteria were identified and subsequently included in our study. The comprehensive forward snowballing process, encompassing both the forward snowballing of included studies and the examination of studies cited in Evans' book, resulted in a total of 23 studies being incorporated into our analysis.

We also performed forward snowballing following Wohlin et al. guidelines (Wohlin, 2014), we used Google Scholar to identify studies that cited our included primary studies. We assessed these additional studies based on their title, abstract, and keywords, using the exclusion criteria established earlier. In cases where a definitive resolution was not achievable based on the provided information, we conducted a rapid analysis of the entire paper, as suggested by Wohlin (2014). Through this process, we identified 8 studies that did not meet any of the exclusion criteria and were therefore included in our study.

3.4.4. Quality assessment

During stage 4 of our primary study identification and selection process, we proceeded with the task of conducting a comprehensive quality assessment of the selected studies. Adhering to the established guidelines put forth by Kitchenham and Charters (2007), each study was meticulously evaluated, with ratings assigned on a scale ranging from 0 to 1. On this scale, a rating of 1 denoted a positive affirmation, 0.5 indicated a partial fulfillment of the quality criteria, and 0 represented a negative response. The specific quality assessment questions utilized in this evaluation can be found in Table 3. Studies that achieved a total score of 4 or higher were deemed to have successfully met the quality assessment criteria and were subsequently included in the data extraction stage. In order to maintain the rigor and objectivity of the process, we adopted a multi-assessor strategy during the quality assessment stage. Each study was independently evaluated by two assessors.

Table 3

List of quality criteria adopted from Kitchenham et al. guidelines (Kitchenham and Charters, 2007). Each criterion is rated on a scale of 0 to 1. Studies that achieved a score of ≥ 4.0 advanced to the data extraction stage.

#	Quality criteria
QC1	The aim of the study is clearly stated.
QC2	The scope and context of the study are clearly defined.
QC3	The study is variable, valid and reliable.
QC4	The research process of the study is documented properly.
QC5	The research questions in the study are answered.
QC6	Negative findings in the study are presented.
QC7	The main findings are clearly stated.
QC8	Conclusions relate to the aim and purpose of the study.

Table 4

Primary study identification metrics showing accessed databases, search dates, number of resulting studies, conflicted studies, included studies, and the count of studies that passed quality checks.

Database	Search date	Results	Conflict	Included	Quality C.
ACM	March 2023	153	0	13	12
IEEE Xplore	March 2023	19	0	8	3
SpringerLink	March 2023	510	1	8	5
Wiley	March 2023	25	0	2	1
Sum Database		707	1	31	21
Scopus	April 2023	608	10	15	9
Google Scholar	June 2023	93	0	8	6
Sum Snowball		701	10	23	15
Total		1408	11	54	36

In cases of disagreement or doubt, the assessors consulted with each other to reach a consensus. This approach ensured that the quality assessments were rigorous, minimized potential biases, and allowed discrepancies to be resolved systematically. After quality assessment, we identified the 36 primary studies. The final list of primary studies can be found in Table 6.

Among the 36 selected studies, the majority demonstrated strong adherence to methodological rigor: 78% received a full score (1.0) for clearly stating research aims (QA1), and 75% scored full on explaining their methodology (QA2). However, some common weaknesses were observed. For instance, 39% of studies scored partially (0.5) or zero on reporting negative findings (QA5), indicating a tendency to report only positive outcomes. Additionally, 36% had incomplete descriptions of their data collection or analysis procedures (QA3), which limits reproducibility. These gaps reflect typical challenges in empirical DDD research, where reporting standards may vary, and certain studies lean more toward industrial experience than academic evaluation. Despite this, all selected studies met a minimum quality threshold, ensuring a consistent baseline for analysis.

3.5. Data extraction and synthesis

3.5.1. Data extraction

In order to ensure consistency and alignment with our research questions and goals, a data extraction form was developed (Table 5). The initial list of data points was prepared by the first author, and this list was then discussed among the authors to refine and finalize a starting set of categories. A pilot data extraction was done on a subset of randomly selected primary studies. During the data extraction process, we adopted an iterative and incremental approach, continuously refining the categories as new insights emerged. This iterative refinement allowed us to adapt the data extraction form to better capture the nuances and specific details relevant to our RQs and goals.

The data points were categorized into 3 main categories. Firstly, the *Publication Metadata* category involved extracting basic information such as the title, authors, and publisher of each study. Secondly, the

Table 5

Data extraction form.

Category	Field	Input type	RQ
<i>Publication Metadata</i>	ID	Auto Alphanumeric	–
	Title	Free Text	RQ1
	Author	Free Text	RQ1
	Item Type	Selection	RQ1
	Database	Selection	RQ1
	Publication Year	Number	RQ1
	Publisher	Free Text	RQ1
	Source	Selection	RQ1
<i>Study Characteristics</i>	Study Design	Selection	RQ1
	Data Collection Methods	Free Text	RQ1
	Academic/Industry	Binary	RQ1
	Participants	Free Text	RQ7
	Empirically Evaluated	Binary	RQ1
	Human Metrics	Binary	RQ1
	Evaluation Methods	Free Text	RQ6
	Evaluation Metrics	Free Text	RQ6
<i>DDD (Context)</i>	Evaluation Results	Free Text	RQ6
	Software System	Free Text	RQ2
	Problems/Motivations	Free Text	RQ3
	Evaluation of Effectiveness	Free Text	RQ6
	Challenges and Limitations	Free Text	RQ5
<i>DDD (Content)</i>	Lessons Learned	Free Text	RQ4-5
	Ubiquitous Language	Binary	RQ4
	Bounded Context	Binary	RQ4
	ACL	Binary	RQ4
	Aggregation	Binary	RQ4
	Domain Events	Binary	RQ4
	Domain Services	Binary	RQ4
	Entities and Value Objects	Binary	RQ4

Study Characteristics category included data related to the study design, data collection methods, and evaluation metrics employed. Lastly, the *Domain-Driven Design* category focused on extracting information specifically pertaining to the application of DDD in each study.

Following the completion of the quality assessment stage, the final set of studies deemed relevant and has sufficient quality to be moved to the data extraction stage. The data extraction process was carried out by the first author using a predefined data extraction form. This form served as a structured tool to systematically collect and record pertinent information from the selected studies.

3.5.2. Synthesis

In this study, the data extraction process involved the collection of both qualitative and quantitative data from the final set of primary studies to derive our results. Specifically, for research questions (RQs) encompassing 1 to 4, we focused on extracting quantitative data. The frequencies and percentages of each identified category were then computed and reported to address these specific research questions. This approach allowed us to quantify and present the distribution and prevalence of different factors and concepts relevant to the research questions under consideration.

Conversely, for RQs pertaining to RQs 5 to 7, we concentrated on extracting qualitative data from the primary studies. To synthesize and interpret these qualitative findings, we applied a narrative synthesis method based on the guidance provided by Popay et al. (2006). This involved systematically extracting key information and important findings from each individual study. Subsequently, we conducted a comprehensive analysis and interpretation of the extracted qualitative data to draw overarching and meaningful conclusions in response to the respective research questions. The narrative synthesis approach allowed us to integrate diverse qualitative evidence, explore patterns and themes across the studies, and provide a cohesive and comprehensive understanding of the research questions in question.

Table 6

List of primary studies and their quality assessment scores.

	Title	Author(s)	Year	Score
Singjai et al. (2021b)	Patterns on Deriving APIs and their Endpoints from Domain Models	Singjai et al.	2021	4
Maddodi et al. (2020)	Aggregate Architecture Simulation in Event-Sourcing Applications using Layered Queuing Networks	Maddodi et al.	2020	7.5
Peng and Hu (2007)	IAnticorruption: a domain-driven design approach to more robust integration	Peng et al.	2007	4.5
Josélyne et al. (2018)	Partitioning microservices: a domain engineering approach	Josélyne et al.	2018	5
Perillo et al. (2009)	Daileon: a tool for enabling domain annotations	Perillo et al.	2009	4.5
Le et al. (2019)	jDomainApp: A Module-Based Domain-Driven Software Framework	Le et al.	2019	6.5
Le et al. (2022)	Generating Multi-platform Single Page Applications: A Hierarchical Domain-Driven Design Approach	Le et al.	2022	5
Uludağ et al. (2018)	Supporting Large-Scale Agile Development with Domain-Driven Design	Uludağ et al.	2018	7
Wang et al. (2022)	A Reference Architecture for Blockchain-based Traceability Systems Using Domain-Driven Design and Microservices	Wang et al.	2022	6.5
Landre et al. (2006)	Architectural improvement by use of strategic level domain-driven design	Landre et al.	2006	6.5
Kapferer and Zimmermann (2020)	Domain-Driven Service Design: Context Modeling, Model Refactoring and Contract Generation	Kapferer et al.	2020	5.5
Camilli et al. (2023)	Actor-driven Decomposition of Microservices through Multi-level Scalability Assessment	Camilli et al.	2023	7
Braun et al. (2021b)	Tackling Consistency-related Design Challenges of Distributed Data-Intensive Systems: An Action Research Study	Braun et al.	2021	7
Landre et al. (2007)	Agile enterprise software development using domain-driven design and test first	Landre et al.	2007	6.5
Braun et al. (2021a)	Advanced Domain-Driven Design for Consistency in Distributed Data-Intensive Systems	Braun et al.	2021	6.5
Zhao and Zhao (2021)	Applying Microservice Refactoring to Object-Oriented Legacy System	Zhao et al.	2021	4
Krause et al. (2020)	Microservice Decomposition via Static and Dynamic Analysis of the Monolith	Krause et al.	2020	5.5
Snoeck and Wautelet (2022)	Agile MERODE: a model-driven software engineering method for user-centric and value-based development	Snoeck et al.	2022	6
Danenas and Garsva (2012)	Domain Driven Development and Feature Driven Development for Development of Decision Support Systems	Danenas et al.	2012	4
Kapferer and Zimmermann (2021)	Domain-Driven Architecture Modeling and Rapid Prototyping with Context Mapper	Kapferer et al.	2021	6.5
Hammarström and Herzog (2016)	Experience from integrating Domain Driven Software System Design into a Systems Engineering Organization	Hammarström et al.	2016	5.5
Da Silva et al. (2022)	BPM2DDD: A Systematic Process for Identifying Domains from Business Processes Models	Da Silva et al.	2022	6.5
Hippchen et al. (2017)	Designing Microservice-Based Applications by Using a Domain-Driven Design Approach	Hippchen et al.	2017	6
Ding et al. (2020)	Enterprise service application architecture based on Domain Driven Model Design	Ding et al.	2020	5
Özkan et al. (2023)	Refactoring with domain-driven design in an industrial context: An action research report	Özkan et al.	2023	8
Pereira and Silva (2022)	Towards Transactional Causal Consistent Microservices Business Logic	Pereira et al.	2022	6.5
Joselyne et al. (2021)	A Systematic Framework of Application Modernization to Microservice based Architecture	Joselyne et al.	2021	4.5
Wanderley and Da Silveria (2012)	A Framework to Diminish the Gap between the Business Specialist and the Software Designer	Wanderley et al.	2012	5
Koryl (2017)	Active resources concept of computation for enterprise software	Koryl et al.	2017	4
Mayer and Weinreich (2018)	An Approach to Extract the Architecture of Microservice-Based Software Systems	Mayer et al.	2018	5
Vural and Koyuncu (2021)	Does Domain-Driven Design Lead to Finding the Optimal Modularity of a Microservice?	Vural et al.	2021	8
Oukes et al. (2021)	Domain-Driven Design applied to land administration system development: Lessons from the Netherlands	Oukes et al.	2021	4.5
Häser et al. (2016)	Is business domain language support beneficial for creating test case specifications: A controlled experiment	Häser et al.	2016	6.5
Le et al. (2018)	On domain driven design using annotation-based domain specific language	Le et al.	2018	6
Abeck et al. (2019)	A Context Map as the Basis for a Microservice Architecture for the Connected Car Domain	Abeck et al.	2019	4
Bünder and Kuchen (2019)	A model-driven approach for behavior-driven GUI testing	Bünder et al.	2019	5.5

4. Results

This section presents a comprehensive analysis of the findings obtained from our SLR conducted on DDD and aims to address the research questions by providing a detailed exploration of the current state of research pertaining to DDD in the literature. Through a meticulous examination of various articles, studies, and sources, the section

sheds light on the key insights and trends related to the implementation, effectiveness, and assessment of DDD in diverse software systems. By organizing the results around the research questions, this section offers valuable insights into the involvement of key stakeholders, the evaluation and measurement of DDD's effectiveness, and the overarching research landscape concerning this domain-centric architectural approach.

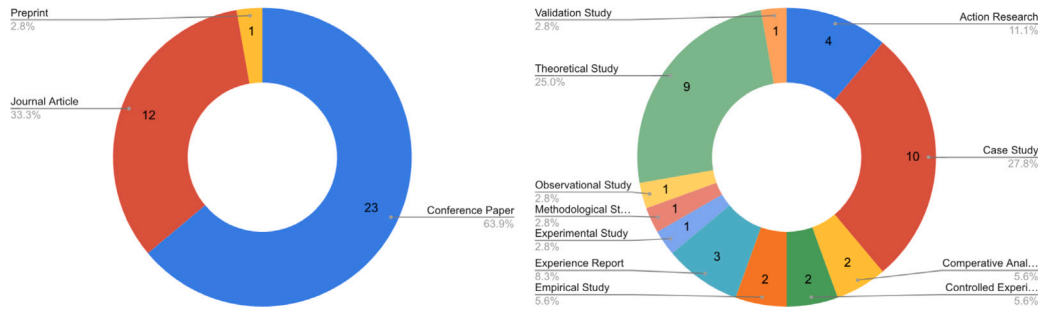


Fig. 2. Breakdown of study types (left) and study designs (right).

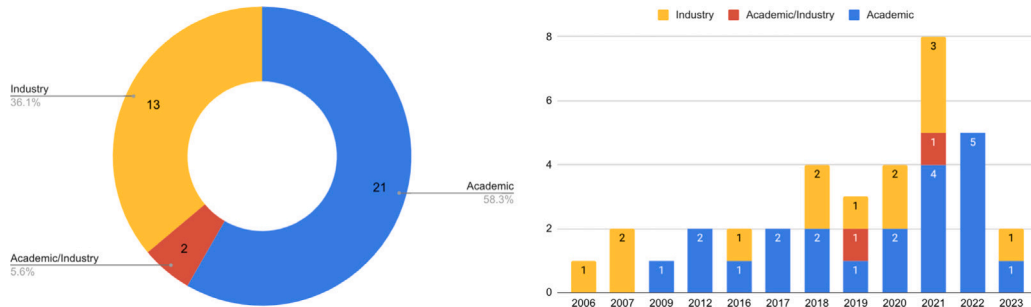


Fig. 3. Breakdown of the primary studies based on their field of focus (left) and field of focus by publication year (right).

4.1. (RQ1) What is the state of the art concerning DDD in the existing literature, in terms of study types, research venues, software systems, and the distribution between academic and industrial settings?

Based on our primary studies (36), the current state of research concerning DDD in the existing literature showcases studies conducted between 2006 and 2023. Among these studies, 23 (64%) are *Conference Papers*, 12 (33%) are *Journal Articles*, and 1 (3%) study is from the *Preprint* database, which also can be seen in Fig. 2.

In terms of venues, a diverse range of conferences and journals have emerged as platforms for the dissemination of DDD research. These venues span the spectrum of software engineering and technology domains. Among them, the *Conference on Object-Oriented Programming Systems, Languages, and Applications* takes a prominent role with 3 publications, showcasing its relevance in fostering discussions around object-oriented paradigms in software development. Additionally, the *Symposium on Information and Communication Technology* have contributed 2 publications.

The most common study type is *Case Study*, accounting for 10 (28%) studies, followed by *Theoretical Study* with 9 (25%) studies. There are also 4 (11%) *Action Research* studies, 3 (8%) *Experience Reports*, 2 (6%) *Empirical Studies*, 2 (6%) that are not explicitly mentioning a type but using empirical methodologies, *Controlled Experiments*, 2 (6%) *Comparative Analyses*, 1 (3%) *Observational Study*, 1 (3%) *Methodological Study*, and 1 (3%) *Experimental Study*. The remaining studies were primarily conceptual, methodological, or design proposals with limited or no empirical validation.

In terms of measurement rigor, 17 studies did not report any concrete evaluation metrics. Those studies are theoretical or design-oriented. Many propose methods, tools, or architectural patterns without validating them through empirical means. While some are labeled as case studies or experience reports, their evidence is often anecdotal and lacks systematic measurement. This pattern suggests a gap in empirically grounded research in certain areas of DDD, particularly tool development and architectural modeling.

When cross-referencing these findings with the application domains, we observed that empirical evaluations were more likely to be found in

industrial case studies involving microservices and distributed systems, while studies focused on tools or frameworks tended to lack rigorous empirical assessment. Furthermore, studies based on action research or industry collaborations generally provided more detailed insight into DDD's impact on software quality, maintainability, and organizational communication.

Regarding the software systems studied, *Microservices* received the highest attention with 16 (44%) studies, followed by *Enterprise Software Systems* with 6 (17%) studies, and DDD-related *Tools* and *System Agnostic* studies with 4 (11%) studies each. Additionally, *Standard Computer Applications* and *Web Applications* were subjects of 2 (6%) studies each, while *Service-Oriented Applications* and *Distributed Systems* each had 1 (3%) study (see Fig. 3).

Out of the 16 *Microservices* studies, 10 (63%) were conducted in an academic environment, while the remaining 6 (38%) were performed in industrial settings. In contrast, *Enterprise Software Systems* were predominantly studied in industry settings. Interestingly, DDD-related *Tools* were exclusively implemented in academic settings.

The interest in DDD initially emerged in industry settings, as evidenced by the first 3 studies conducted between 2006 and 2007. Subsequently, the focus shifted to academic settings, with all 3 studies from 2009 to 2012 being conducted in academia. The trend then became more balanced, with an equal distribution of studies between academic and industrial settings in 2016.

The distribution of studies across academic and industry settings reveals an interesting pattern. Out of the total 36 studies, 21 (58.3%) were conducted in academia, indicating a dominant presence of academic research in this field. Conversely, 13 (36.1%) studies were conducted in industry settings, and 2 (5.6%) studies had a hybrid setting involving both academic and industry collaboration. This disparity in the academic-industry ratio suggests that DDD has garnered significant attention from the academic community, where researchers are actively exploring its concepts and applications. However, it also highlights the relevance and practical implications of DDD in industry settings, as evidenced by the notable number of industry-based studies.

The existing literature demonstrates a diverse and evolving landscape of research on DDD, with a particular emphasis on *Microservices*

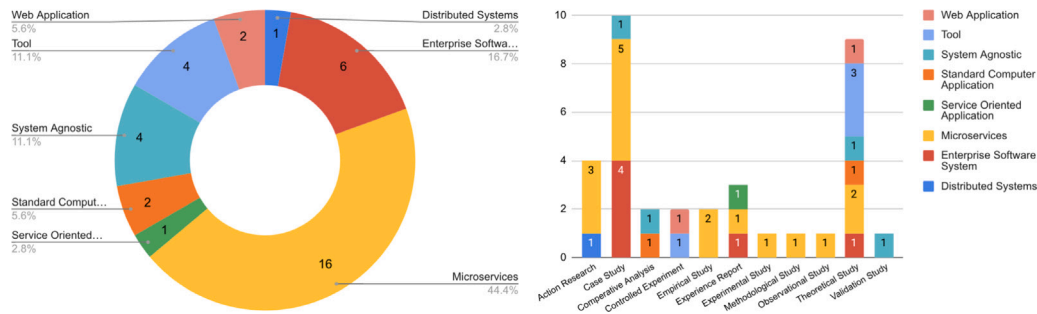


Fig. 4. Breakdown of the primary studies based on applied software systems (left) and studies grouped by the study type and software systems (right).

and an increasing number of academic contributions. This suggests a growing recognition of the relevance and applicability of DDD in both academic and industrial contexts.

4.2. (RQ2) What categories of software systems have demonstrated benefits from the implementation of DDD?

With this RQ, we present the categories of software systems that have demonstrated benefits from the implementation of DDD, based on the findings. The studies were analyzed, and their contributions to various software system categories were identified during the data extraction phase. The breakdown of the following data can also be visually seen in Fig. 4 and Table 7.

4.2.1. Microservices

Several studies have highlighted the advantages of applying DDD principles in the context of microservices architecture (Singjai et al., 2021b; Josélyne et al., 2018; Wang et al., 2022; Camilli et al., 2023; Braun et al., 2021a; Zhao and Zhao, 2021; Krause et al., 2020; Hippchen et al., 2017; Ding et al., 2020; Özkan et al., 2023; Pereira and Silva, 2022; Joselyne et al., 2021; Mayer and Weinreich, 2018; Vural and Koyuncu, 2021; Oukes et al., 2021; Abeck et al., 2019). According to our learnings from those studies, by adopting DDD, organizations have been able to improve the modularity, maintainability, and scalability of microservices-based applications, enabling effective handling of complex and distributed systems. The use of DDD patterns and concepts in the design and development of microservices has been particularly beneficial in achieving clear boundaries between services, enabling better communication between development teams, and ensuring a strong alignment with business domains. For instance, Krause et al. (2020) applied DDD principles to modernize a real-world lottery system into a microservices-based architecture, demonstrating the approach's relevance in refactoring legacy systems in the public sector.

4.2.2. Enterprise software systems

Several studies have reported the benefits of applying DDD in the context of enterprise software systems (Maddodi et al., 2020; Peng and Hu, 2007; Uludağ et al., 2018; Landre et al., 2007; Danenas and Garsva, 2012; Koryl, 2017). The adoption of DDD principles has proven valuable in handling the increasing complexity of enterprise systems and addressing challenges related to legacy system modernization. By emphasizing bounded contexts and domain models, DDD enables more maintainable and flexible enterprise software architectures, allowing organizations to better adapt to changing business requirements.

4.2.3. Web applications

Some studies have shown that DDD can be beneficial in the development of web applications (Le et al., 2022; Bänder and Kuchen, 2019). By applying DDD principles, web application developers have been able to improve the design and implementation of Single Page Applications (SPAs), achieving better cross-framework compatibility and enhancing productivity in SPA development.

4.2.4. Tools and standard computer applications

Several studies have applied DDD in the context of tools (Perillo et al., 2009; Kapferer and Zimmermann, 2020; Häser et al., 2016) and standard computer applications (Le et al., 2019; Hammarström and Herzog, 2016). DDD has been leveraged to improve code quality, enhance the software construction process, and provide more comprehensive solutions for domain modeling and software generation. In the case of tools, DDD has been employed to bridge the gap between business domain concerns and infrastructure, ensuring the maintainability and readability of code.

4.2.5. Distributed systems

In this study, we classified systems based on the architectural types explicitly mentioned in the primary studies. However, some system types, such as Service-Oriented Architecture (SOA) (Section 4.2.6) and Distributed Systems, may overlap in practice. For example, SOA is one possible way of realizing a distributed system.

In cases where the primary study did not specify the exact architectural type such as microservices or SOA, but specifies it is a distributed system, we used the more generic term Distributed Systems to accurately reflect the nature of the system without over-specification.

This classification ensures that we do not make assumptions about the specific system architecture when such information is not available, and it provides a comprehensive view of how DDD is applied in various types of distributed systems. We acknowledge that system types like SOA and Distributed Systems are not always mutually exclusive, and we carefully categorized the systems according to the information available in the primary studies.

This approach was applied to Braun et al.'s study (Braun et al., 2021b), where it reported that DDD has demonstrated benefits in the development of distributed systems. By controlling concurrent data access and achieving semantic compatibility in domain models, DDD has been instrumental in tackling consistency-related design challenges in distributed data-intensive systems. Similarly, Wang et al. (2022) utilized DDD to architect a blockchain-based traceability system in the global supply chain domain, where managing bounded contexts across distributed actors was critical.

4.2.6. Service-Oriented Applications (SOA)

Landre et al. (2006) use DDD patterns to improve the software architecture of a large enterprise system, which is designed as a SOA. The authors mention that establishing a DDD context map helped in scoping the project and identifying architectural problems such as complex dependencies and gateway roles.

In this study, we distinguish between SOA and Microservices as separate architectural classifications, not to make a strict ontological separation, but to reflect the terminology used in the primary studies themselves. While both paradigms promote modular and service-based decomposition, they also differ in typical implementations: microservices emphasize lightweight, independently deployable components with decentralized governance, whereas SOA is often realized

with centralized orchestration and heavier protocols (e.g., ESB and SOAP) (Dragoni et al., 2017; Fritzsche et al., 2019).

We acknowledge that this boundary is not universally agreed upon, and many researchers recognize significant overlap between SOA and microservices. As noted by Lewis and Fowler (2014), the distinction lies more in implementation philosophy than fundamental principles. Our categorization aims to reflect how the systems were described and studied in the reviewed literature, without implying that SOA and microservices are categorically distinct. Future work could explore how these architectural paradigms co-evolve in practice.

4.2.7. System agnostic

Certain studies have shown that DDD can provide valuable contributions for the implementation of software which is suitable across various system categories (Snoeck and Wautelet, 2022; Kapferer and Zimmermann, 2021; Da Silva et al., 2022; Wanderley and Da Silva, 2012). In these studies, DDD principles and patterns have been applied to implement practitioner frameworks and methods to address communication gaps between business specialists and software designers, improve software system modularity, and support the creation of context maps.

4.2.8. DDD tools

DDD has been applied to improve code quality, readability, and maintainability through the development of supportive tooling. These tools fall into two main categories. Annotation-based DDD frameworks, such as Daileon by Perillo et al. (2009), aim to separate business domain concerns from infrastructure code. In contrast, DDD modeling tools, such as Context Mapper by Kapferer and Zimmermann (2020), provide higher-level support for visualizing and evolving Context Maps and other domain artifacts.

4.3. (RQ3) Which specific software development concerns have been addressed through the application of DDD in previous studies?

The SLR conducted in this study sought to identify the specific software development concerns that have been addressed through the application of DDD in previous research. The results revealed several challenges and motivations encountered in various software development projects, encompassing diverse domains such as microservices architecture, legacy system modernization, distributed systems, and complex decision support systems. The subsequent information present a comprehensive overview of the identified challenges and motivations for adopting DDD.

4.3.1. Addressing API design problems in distributed systems

One of the prominent challenges addressed by DDD in software development projects is related to API design in distributed systems, particularly in microservice-based systems. This challenge revolves around missing guidance on how to derive APIs and API endpoints from domain model elements. The primary concern is that coupling smells lead to poor API design in distributed systems. These issues result in shallow APIs that are difficult to understand, maintain, and evolve. Singjai et al. (2021b) propose clear guidance on how to derive APIs and API endpoints from domain model elements using DDD patterns.

4.3.2. Investigating performance in component-based software

Command Query Responsibility Segregation (CQRS) and Event Sourcing (ES) are two architectural patterns often used in conjunction with DDD to improve scalability and traceability. CQRS separates write operations (commands) from read operations (queries), enabling systems to optimize each independently. Event Sourcing, on the other hand, persists all changes to the application state as a sequence of immutable events rather than storing the current state directly. In this approach, the current state is reconstructed by replaying these events.

These patterns align well with DDD's tactical building blocks — such as aggregates and domain events — and have been leveraged in studies to evaluate performance characteristics under different workload patterns (Maddodi et al., 2020).

In pursuit of achieving optimized performance and resource metrics in component-based software, DDD has been used in research to investigate the effect of workload patterns. Specifically, the research aimed to gain insight into how patterns like CQRS and ES are created, thus providing valuable guidelines for architects to select appropriate architectures based on workload characteristics (Peng and Hu, 2007).

4.3.3. Reducing complexity in currency exchange systems

A prevalent software development problem addressed by DDD pertains to reducing complexity and high coupling in currency exchange systems. The case in question involves the integration layer between a new currency exchange system (SPOT) and a legacy back office system (TBS), wherein the integration layer (TBSEExport) had become an unwieldy burden with unpredictable side effects following minor changes. To tackle this complexity and streamline the integration process, researchers sought to devise effective solutions using DDD principles (Peng and Hu, 2007).

4.3.4. Microservice size, partitioning and defining API endpoints

Microservice architecture introduces the challenge of determining the appropriate size of individual microservices, thereby necessitating a conceptual methodology for partitioning microservices based on domain engineering techniques. Improperly sized microservices can lead to complexity management issues, where too large services become monolithic and too small services create an overly complex system. This can also result in performance bottlenecks and scalability challenges, as well as data consistency problems due to poorly coordinated related data spread across services. Additionally, undefined or poorly defined boundaries complicate maintenance and evolution, and misaligned microservice boundaries with team boundaries hinder team autonomy and require excessive coordination. DDD has been instrumental in addressing this challenge, aiming to achieve optimal microservice design and composition within the context of a microservices architecture (Singjai et al., 2021b; Josélyne et al., 2018).

4.3.5. Improving code quality with domain annotations

In the pursuit of enhancing code quality, readability, and maintainability while ensuring a clear separation between the business domain and infrastructure concerns, DDD has been leveraged to address challenges associated with using annotations in DDD. The proposed concept of “domain annotations” by Perillo et al. (2009) serves to mitigate these challenges and fosters a robust software design approach.

4.3.6. Enhancing software construction and development environment

The lack of generative, module-based software construction and seamless development environment integration poses significant challenges in software development. Without these capabilities, developers face difficulties in automating the generation of software from domain models, leading to increased manual effort and potential for errors (Le et al., 2019). This can result in inconsistencies between the domain model and the implemented software, hindering maintainability and scalability. Additionally, the absence of integrated development tools means that developers must switch between different environments and tools, disrupting their workflow and reducing productivity. By addressing these issues, in the work of Le et al. (2019) the adoption of the jDomainApp framework and an Eclipse IDE plugin has been proposed as a solution to these challenges, thereby optimizing software construction and development processes.

4.3.7. Multi-platform SPA development

The challenges faced by Single Page Application (SPA) developers encompass two crucial aspects: designing SPAs that can effectively work across different SPA frameworks (e.g., Angular, React, React Native, and Vue.js) and translating this design into an intermediate high-level language that can seamlessly transform into a target framework of choice. DDD offers a multi-platform, hierarchical approach to tackle these challenges, significantly enhancing SPA development productivity (Le et al., 2022).

4.3.8. Scaling agility in large organizations

Large organizations encounter a myriad of challenges when striving to scale agility, including effective coordination and communication between agile teams, inter-team dependencies, and the need for clearly defined requirements. By adopting DDD, enterprises have sought to address these challenges and provide architectural guidance to support large-scale agile development. However, it is important to acknowledge that successful DDD implementation in large organizations necessitates experienced enterprise architects to provide effective guidance (Uludağ et al., 2018).

4.3.9. Complexities in Blockchain-Based Traceability Systems (BTS)

BTSs are software systems designed to ensure transparent, tamper-proof tracking of products or data throughout a supply chain or process. These systems leverage blockchain technology to record transactions and state changes in an immutable ledger, enhancing trust, provenance, and accountability.

In the context of DDD, BTSs benefit from patterns such as Bounded Contexts and Aggregates to model complex interactions and responsibilities across different stakeholders and subsystems. As reported in Wang et al. (2022), the application of DDD in BTS improves the maintainability and extensibility of these systems by clearly delineating domain boundaries and aligning technical implementations with business processes.

The design and development of BTSs pose unique challenges and complexities, including domain model intricacies, modeling boundaries, and managing the relationship between different bounded contexts. Researchers have employed DDD and microservice architecture to address these challenges, thereby improving the cohesiveness, maintainability, and extensibility of BTSs in handling complex data in the global supply chain in Wang et al. (2022).

4.3.10. Challenges in model decomposition

Decomposing software systems into cohesive and loosely coupled modules poses significant challenges for software engineers and service designers. DDD patterns have been instrumental in addressing this challenge, offering a clear and concise interpretation of these patterns, despite debates and differing interpretations among practitioners (Kapferer and Zimmermann, 2021).

4.3.11. Legacy system modernization and refactoring

Legacy systems often suffer from outdated technologies, poor documentation, and a lack of alignment between the system's current state and the evolving business requirements. This misalignment creates a communication gap between business specialists, who understand the domain requirements, and software designers, who are tasked with implementing these requirements (Özkan et al., 2023). As a result, changes and enhancements become risky, time-consuming, and error-prone. DDD addresses these issues by providing a structured approach to align the domain model with business needs, facilitating better communication and understanding. This alignment enables more effective refactoring and modernization efforts, ensuring that the updated system meets current business requirements while maintaining or improving system integrity and performance (Özkan et al., 2023; Wanderley and Da Silveria, 2012).

Table 7

Reported benefits of DDD across different software system categories, as identified in the primary studies.

Software system category	Reported benefits of DDD
Microservices	Improved modularity, maintainability, and scalability; better service boundary definition; enhanced team autonomy; reduced coupling through bounded contexts and context mapping (Singjai et al., 2021b; Josélyne et al., 2018; Wang et al., 2022; Camilli et al., 2023).
Enterprise Software Systems	Support for legacy system modernization; clearer architecture via bounded contexts; improved adaptability to evolving business requirements (Maddodi et al., 2020; Uludağ et al., 2018; Landre et al., 2007).
Web Applications	Better SPA design across frameworks; productivity boost through DDD-driven component generation (Le et al., 2022; Bünder and Kuchen, 2019).
Tools	Improved code quality and modularity through domain annotations; enhanced maintainability by separating domain logic from infrastructure concerns (Perillo et al., 2009; Kapferer and Zimmermann, 2020).
Distributed Systems	Addressed consistency challenges; improved semantic compatibility and isolation between modules (Braun et al., 2021b).
SOA/Service-Oriented Applications	Clear project scoping; identification of architectural problems (e.g., dependency complexity) using context maps (Landre et al., 2006).
System-Agnostic Approaches	Support for agile methods; communication alignment between business and technical stakeholders; improved modeling modularity (Snoeck and Wautelet, 2022; Da Silva et al., 2022).

4.4. (RQ4) What are the most common DDD patterns utilized in previous research, such as ACLs, Bounded Contexts, and other related approaches?

In this SLR on DDD, we investigated the most commonly utilized DDD patterns across various software systems. The analysis was based on a dataset comprising studies identified through our study selection process explained in Section 3.4. Each study was examined for the presence or absence of specific DDD patterns, marked by “Y” for used and “N” for not used patterns, during our data extraction process. Our results are also visually shown in Fig. 5.

The results reveal that certain DDD patterns are prevalent in the majority of the analyzed studies, indicating their significance and widespread adoption in the DDD context. The following DDD patterns were found to be most commonly utilized:

1. **Ubiquitous Language.** Ubiquitous language is a fundamental DDD pattern, and our findings indicate that it is frequently employed in various software systems, including Microservices, Enterprise Software Systems and Web Applications (Maddodi et al., 2020; Peng and Hu, 2007; Josélyne et al., 2018; Le et al., 2022; Uludağ et al., 2018; Krause et al., 2020; Danenas and Garsva, 2012; Kapferer and Zimmermann, 2021; Özkan et al., 2023; Pereira and Silva, 2022; Joselyne et al., 2021; Oukes et al., 2021).
2. **Bounded Context.** Bounded contexts were found to be widely utilized in Microservices, Enterprise Software Systems and Web Applications (Maddodi et al., 2020; Peng and Hu, 2007; Josélyne et al., 2018; Le et al., 2022; Uludağ et al., 2018; Camilli et al., 2023; Braun et al., 2021b,a; Zhao and Zhao, 2021; Snoeck and Wautelet, 2022; Kapferer and Zimmermann, 2021; Özkan et al., 2023; Pereira and Silva, 2022).

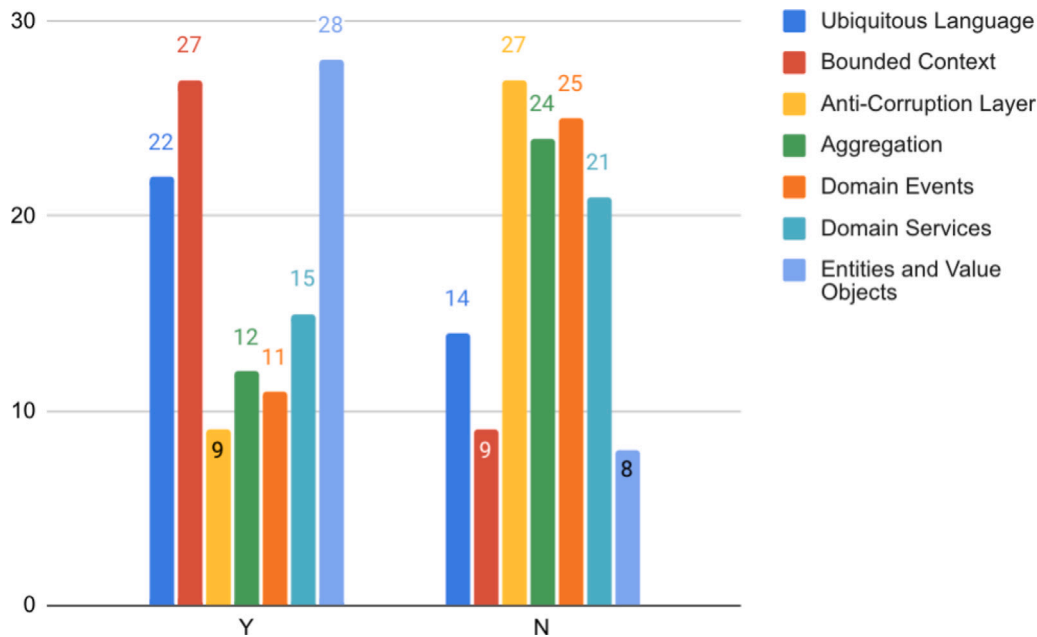


Fig. 5. Breakdown of the applied DDD patterns in the primary studies. Applied DDD patterns marked with “Y” and not applied pattern marked with “N”.

3. *Entities and Value Objects*. Entities and value objects, representing domain concepts, were extensively adopted in Microservices, Enterprise Software Systems and Web Applications (Maddodi et al., 2020; Peng and Hu, 2007; Josélyne et al., 2018; Le et al., 2022; Uludağ et al., 2018; Wang et al., 2022; Braun et al., 2021b; Snoeck and Wautelet, 2022; Danenas and Garsva, 2012; Kapferer and Zimmermann, 2021; Hammarström and Herzog, 2016; Da Silva et al., 2022; Hippchen et al., 2017; Özkan et al., 2023; Pereira and Silva, 2022; Joselyne et al., 2021; Wanderley and Da Silveria, 2012).

In contrast, the ACL was identified as the least used DDD pattern across different software systems. Several studies did not utilize the ACL in their respective projects (Peng and Hu, 2007; Le et al., 2019; Landre et al., 2006; Kapferer and Zimmermann, 2020; Koryl, 2017; Häser et al., 2016; Bünder and Kuchen, 2019).

The ACL serves as a means to protect the integrity of the main domain from external and legacy systems, and its limited adoption suggests that certain software projects may have addressed integration challenges through alternative approaches or may not have encountered a specific need for this pattern.

While Ubiquitous Language, Bounded Context, and Entities/Value Objects were widely observed across studies, patterns such as Core Domain and Shared Kernel were rarely reported. Only a limited number of studies (e.g., Landre et al. (2006), Özkan et al. (2023)) referred to the Core Domain explicitly, often in the context of prioritizing critical business logic. Shared Kernel was not discussed as a standalone concept in any study, although collaboration between teams was sometimes implied in studies using Bounded Contexts. Similarly, upstream-downstream relationships — central to context mapping in Strategic Design — were briefly mentioned in a few cases but not analyzed in depth.

4.5. (RQ5) What are the challenges encountered during the implementation of DDD in software development projects?

The challenges encountered during the implementation of DDD in software development projects are diverse and demand careful consideration to ensure successful adoption. In this RQ, we present the key challenges identified through primary studies. To support the narrative

discussion, we also summarize the main DDD implementation challenges and mitigation strategies identified across the primary studies. Table 8 and Table 9 presents a concise mapping that highlights both the barriers encountered and the approaches taken to address them in practice.

4.5.1. Addressing design complexities

Several studies highlight the complexity associated with implementing DDD patterns, particularly in the context of microservices and distributed systems. The integration layer between different Bounded Contexts can become burdensome, leading to high coupling and unpredictability in the system. This challenge, for instance, is exemplified in the case of the currency exchange system (SPOT) and the legacy back-office system (TBS) in Peng and Hu (2007). Finding the right balance between high cohesion within services and loose coupling between them is crucial for scalability and maintainability (Peng and Hu, 2007; Josélyne et al., 2018).

4.5.2. Identifying optimal microservices boundaries

Decomposing monolithic systems into microservices presents its own set of challenges. Determining the appropriate size and boundaries of microservices is not straightforward and requires careful consideration of the domain and workload characteristics (Josélyne et al., 2018). The granularity of microservices can be subjective, leading to potential disagreements among development teams (Kapferer and Zimmermann, 2021). Ensuring that microservices align with business needs and subdomains can be challenging (Hippchen et al., 2017). Additionally, finding suitable examples that demonstrate DDD-driven microservices architecture is often noted as a limitation (Vural and Koyuncu, 2021).

4.5.3. Managing domain model complexity

When adopting DDD, managing the complexity of the domain model becomes a crucial challenge. Modeling boundaries between different Bounded Contexts and ensuring the relationships between them are properly managed require deep knowledge about the organization's domain (Da Silva et al. (2022)). The complexity of the domain model and the interactions between bounded contexts can hinder the process of abstraction and generalization of elements (Da Silva et al., 2022).

Table 8

Challenges in implementing DDD and corresponding mitigation strategies and techniques reported in the literature.

Challenge	Mitigation strategy and concrete techniques (from literature)
Design complexity in distributed systems	Use of context mapping and bounded contexts to reduce coupling; ACLs to encapsulate legacy interactions (Peng and Hu, 2007; Jos�lyne et al., 2018)
Defining microservices boundaries	Domain decomposition using business-aligned subdomains and context maps; Use of design workshops to iteratively refine service granularity (Jos�lyne et al., 2018; Hippchen et al., 2017; Vural and Koyuncu, 2021)
Managing complex domain models	Collaborative modeling with domain experts; event storming sessions; maintaining cohesion through tactical DDD patterns (Da Silva et al., 2022; Kapferer and Zimmermann, 2020; Wang et al., 2022)
Integration with legacy systems	Use of ACLs to decouple legacy dependencies and allow incremental migration (Peng and Hu, 2007)
Communication gaps between domain experts and developers	Establishing Ubiquitous Language and domain co-design practices; continuous involvement of stakeholders (Wanderley and Da Silveria, 2012; Kapferer and Zimmermann, 2020)
Model-code misalignment in DSL-based tooling	Use of tools like Context Mapper for synchronized modeling and code generation; contract-based validation (Kapferer and Zimmermann, 2020)
Onboarding and lack of DDD expertise	Internal DDD training sessions; mentoring by experienced architects; gradual learning curve through pilot projects (�zkan et al., 2023; Hippchen et al., 2017)
Pattern interpretation disagreements	Team workshops to align on DDD pattern usage; creation of design documentation to guide consistent practices (Kapferer and Zimmermann, 2021; �zkan et al., 2023)
Limited availability of domain experts	Early involvement of domain specialists; use of domain walkthroughs and business process models to extract knowledge (Vural and Koyuncu, 2021; Da Silva et al., 2022)

This challenge is especially pronounced when dealing with large-scale and complex data-intensive systems (Wang et al., 2022).

4.5.4. Integrating DDD with existing frameworks

Integrating DDD into existing software frameworks can present compatibility issues, as DDD introduces encapsulation and abstraction levels that may not align seamlessly with existing structures ( zkan et al., 2023). Refactoring in an actively developed codebase can also prove challenging, especially when engineers lack prior experience with DDD ( zkan et al., 2023).

4.5.5. Communication gap between business specialists and software designers

Incorporating DDD requires effective communication between business specialists and software designers to ensure consistency in requirements and conceptual models (Wanderley and Da Silveria, 2012). Misalignment in communication can lead to discrepancies in the final design and implementation.

Table 9

SWOT analysis of DDD in software research.

Strengths	
S1	Aligns software systems with business domains conceptually.
S2	Widespread use of tactical/strategic patterns (e.g., Ubiquitous Language, Bounded Context, Entities, Aggregates).
S3	Demonstrated benefits in modularity and maintainability, especially in microservices.
Weaknesses	
W1	Limited empirical evaluation in several studies.
W2	Underreporting of strategic patterns (e.g., Shared Kernel, Core Domain, upstream/downstream relations).
W3	High learning curve for newcomers to DDD.
Opportunities	
O1	Growing interest in academia and industry since 2017.
O2	Potential for stronger empirical and comparative research.
O3	Increasing tool support (e.g., DSLs, context-mapping tools).
Threats	
T1	Risk of superficial or inconsistent DDD adoption.
T2	Conceptual disagreements and ambiguity in implementation.
T3	Dependence on domain experts may limit broader adoption.

4.5.6. Model-code gap in domain-specific language (DSL)

A domain-specific language (DSL) is a computer language specialized to a particular application domain (Evans (2004)). Studies that focus on DSL-based DDD implementations point out potential weaknesses, including the existence of a “model-code” gap (Kapferer and Zimmermann, 2020). The lack of a deeper knowledge about the domain may hinder the process of abstraction and generalization, leading to artifacts that may not align with the organization’s domain (Da Silva et al. (2022)).

4.5.7. Impact of developer’s experience and challenges in onboarding

The level of experience and familiarity with DDD concepts among the development team can influence the effectiveness of DDD implementation (Hippchen et al., 2017). Developers with prior experience in DDD are more likely to understand and apply DDD patterns effectively, resulting in better-designed and more maintainable software systems (Hippchen et al., 2017). On the other hand, inexperienced developers may struggle with grasping the complexities of DDD and may not fully utilize the benefits of the approach (Hippchen et al., 2017). Also, introducing DDD to a development team that has little or no prior exposure to DDD can be a challenge ( zkan et al., 2023). Onboarding developers and providing the necessary training and resources to understand DDD concepts and principles can require additional time and effort ( zkan et al., 2023). This challenge can be particularly pronounced in organizations that are transitioning from traditional software development approaches to DDD.

4.5.8. Navigating controversial debates

Controversial debates and differing interpretations exist among practitioners regarding the application and combination of DDD patterns (Kapferer and Zimmermann, 2021;  zkan et al., 2023). Developers may face challenges in deciding which patterns to adopt and how to effectively integrate them into the development process. This can result in disagreements within the team and may require additional effort to reach a consensus.

4.5.9. Need for experienced domain experts

Incorporating DDD requires a deep understanding of the domain and business requirements. Experienced domain experts are valuable in guiding the development team to create accurate and effective domain models (Vural and Koyuncu, 2021). However, the availability of such domain experts may be limited, especially in specialized domains, and this can hinder the DDD adoption process.

4.6. (RQ6) How has the effectiveness of DDD been assessed and measured in prior studies?

The effectiveness of DDD has been assessed and measured in prior studies through various evaluation methods and metrics. It is evident from the data that prior studies have employed a combination of quantitative and qualitative evaluation methods to assess the effectiveness of DDD. These methods are described in more detail in the following subsections.

4.6.1. Performance and resource metrics

In the case of Maddodi et al. (2020) the effectiveness of DDD was measured by evaluating the performance and resource metrics of the software system using Command Query Responsibility Segregation (CQRS) and Event Sourcing (ES) frameworks. The study reported relative error values for throughput and response time for different scenarios.

4.6.2. Feedback from stakeholders

In Uludağ et al. (2018) the effectiveness of DDD practices, such as event storming workshops and domain modeling, was measured through the adoption and use of these practices by agile teams. The study also considered feedback from stakeholders, including Program Managers, Product Owners, and Domain Architects, on the benefits and value of DDD in supporting large-scale agile software development.

4.6.3. Context mapping and complexity reduction

Landre et al. (2006) evaluated the effectiveness of DDD, particularly the strategic part of it, by using context mapping and context relationships to analyze the software architecture and identify complexities. The study demonstrated how DDD provides mechanisms to address the identified problems and improve the quality of the Enterprise Architecture and derived software architectures by applying responsibility layers and improving scoping of projects.

4.6.4. Effectiveness of design guidelines

Braun et al. (2021b) conducted an assessment to determine the efficacy of design guidelines pertaining to distributed data-intensive systems. The evaluation encompassed various metrics, including the proportion of compatible domain operations and the extent of trivial aggregates within the domain model.

4.6.5. Questionnaires and developer perception

The study by Özkan et al. (2023), the efficacy of DDD was evaluated using a combination of qualitative methods, including focus group sessions, semi-structured interviews, and think-aloud protocols. Additionally, a quantitative analysis was conducted using the Technology Acceptance Model (mTAM) questionnaire. The findings revealed that DDD positively impacted software maintainability, as developers perceived it. Similarly, Braun et al. (2021b) undertook an assessment of design guidelines effectiveness by gauging developers' perceptions in their study.

4.6.6. Measuring effectiveness of ECD3 guidelines

In Braun et al. (2021a) the effectiveness of the DDD measured based on Eventually Consistent Domain-Driven Design Guidelines (ECD3), assessed by conducting a comparative analysis between compatible and incompatible domain operations in the redesigned domain models. The implementation of the recommended ECD3 guidelines exhibited a notable enhancement in the proportion of compatible operations within the domain models.

4.6.7. Software generation and complexity analysis

The effectiveness of the DDD-based software generation method proposed in Le et al. (2019) was evaluated by examining its linear time complexity and scalability on real-world software of considerable magnitude, through the utilization of the jDomainApp framework.

4.6.8. Time and resource consumption

In Häser et al. (2016) the effectiveness of including business domain concepts in the DSL was measured by evaluating the perceived quality, creation time, and length of test case specifications.

4.6.9. Granularity and coupling analysis

The study (Vural and Koyuncu, 2021) assessed the effectiveness of DDD in microservices by comparing the coupling and cohesion values of different examples. The study concluded that original examples achieved a good balance between low coupling and acceptable cohesion.

4.6.10. Prototype application

In Krause et al. (2020) the effectiveness of the approach for modernizing legacy lottery applications into microservices was evaluated through its successful application to a real-world lottery application.

4.6.11. Metamodel and design models

Le et al. (2022) aimed to assess the efficacy of DDD in constructing Single Page Applications (SPAs) compatible with multiple platforms. For this purpose, they developed a novel metamodel and implemented a hierarchical approach within the DDD framework. The outcomes of the study revealed that employing the domain model as a central component and adhering to the hierarchical DDD approach yielded favorable design models alongside prosperous generation of SPAs tailored for various platforms.

4.7. (RQ7) Who are the key stakeholders involved and benefited in the implementation of DDD?

In the implementation of DDD, several key stakeholders are involved to ensure its successful application. These stakeholders encompass a diverse set of roles, including software engineers, architects, project managers, and domain experts. Through our SLR, we have identified the following key stakeholders.

4.7.1. Software engineers

Software Engineers play a central role in implementing DDD principles and practices in various software systems. They are responsible for translating the domain model into code, implementing domain-specific logic, and ensuring the alignment of the software architecture with the domain requirements (Singjai et al., 2021b; Maddodi et al., 2020; Peng and Hu, 2007; Snoeck and Wautelet, 2022; Özkan et al., 2023).

4.7.2. Architects

Architects are crucial in the implementation of DDD as they provide strategic guidance and oversight on the overall software design. They collaborate with Software Engineers and other stakeholders to define the high-level architecture, bounded contexts, and domain models that capture the domain knowledge effectively (Maddodi et al., 2020; Kapferer and Zimmermann, 2021; Da Silva et al., 2022).

4.7.3. Project managers

Project managers play a vital role in coordinating DDD implementation efforts. They ensure that DDD practices align with project goals, allocate resources appropriately, and facilitate effective communication between different teams and stakeholders (Peng and Hu, 2007; Uludağ et al., 2018).

4.7.4. Domain experts

Domain experts are individuals who possess in-depth knowledge of the specific business domain for which the software system is being developed. Their active involvement is critical for refining the domain model, identifying key domain concepts, and validating the software's alignment with real-world domain requirements (Singjai et al., 2021b; Uludağ et al., 2018; Da Silva et al., 2022).

In addition to the primary stakeholders mentioned above, the literature also identifies other relevant participants involved in the implementation of DDD. These participants include Quality Assurance (QA) Engineers, DevOps Engineers, UX Designers, Business Analysts, Systems Analysts, Data Modeling Specialists, and end-users (Peng and Hu, 2007; Braun et al., 2021a; Özkan et al., 2023; Oukes et al., 2021; Häser et al., 2016; Bündler and Kuchen, 2019).

Taking our findings in consideration, we identified that the successful implementation of DDD requires close collaboration and communication among these diverse stakeholders to ensure that the developed software system effectively represents the domain's complexity and meets the business needs. The active engagement of software engineers, architects, project managers, and domain experts, along with the contributions from other relevant participants, enables the application of DDD principles to create robust and domain-centric software solutions.

5. Discussions

In this section, we aim to consolidate and evaluate the extensive range of information obtained through our RQs in order to present a comprehensive overview of the effectiveness, challenges, and potential consequences associated with the adoption of DDD in software development. Throughout this section, we will do an examination of the findings by establishing connections across various studies and identifying recurring themes and patterns. It is important to acknowledge that certain aspects of the discussion have already been addressed in Section 4 when answering to the RQs. Nevertheless, within this dedicated section, we will explore further nuances, ramifications, and potential applications attributable to DDD based on the available evidence.

5.1. Adoption and implementation of DDD

Several studies report successful implementations of DDD, showcasing its ability to improve software design, maintainability, scalability, and agility. DDD concepts, such as Ubiquitous Language, Bounded Contexts, Aggregates, Entities, Value Objects, and Domain Events, are commonly utilized to model complex domains effectively. Many researchers emphasize the significance of defining clear domain boundaries and using a common language between domain experts and software designers to ensure effective communication and understanding. Additionally, DDD is frequently integrated with other software development paradigms, such as microservices and event-driven architecture, to achieve better modularity and maintainable systems.

5.2. Ubiquitous language as a core principle of DDD

The data analyzed points out that the Ubiquitous Language stands as a foundational principle of DDD. It ensures a shared and consistent understanding of the domain between stakeholders, including business specialists, software designers, and developers (Evans, 2004). The studies consistently emphasize the importance of Ubiquitous Language in facilitating communication, clarifying domain concepts, and improving collaboration among team members (Singjai et al., 2021b; Maddodi et al., 2020; Josélyne et al., 2018; Perillo et al., 2009; Le et al., 2019, 2022; Uludağ et al., 2018; Danenas and Garsva, 2012; Hippchen et al., 2017; Ding et al., 2020; Özkan et al., 2023; Pereira and Silva, 2022; Wanderley and Da Silveria, 2012; Le et al., 2018).

5.3. Bounded context and decomposition of microservices

Bounded Context is another fundamental concept in DDD, and its significance in microservices architecture is evident in several studies (Maddodi et al., 2020; Peng and Hu, 2007; Josélyne et al., 2018; Uludağ et al., 2018; Braun et al., 2021b; Krause et al., 2020; Pereira and Silva, 2022; Joselyne et al., 2021; Vural and Koyuncu, 2021; Bündler and Kuchen, 2019). The concept of Bounded Context is used to define explicit boundaries where specific domain models apply, allowing the system to be decomposed into manageable, cohesive units. Decomposing a system into microservices based on Bounded Contexts enables easier scaling, maintenance, and independent updating of each microservice.

5.4. Domain services, entities, and value objects as a tactical DDD

Several studies discuss the use of Domain Services, Entities, and Value Objects as part of DDD's tactical patterns (Maddodi et al., 2020; Le et al., 2022; Uludağ et al., 2018; Hippchen et al., 2017; Özkan et al., 2023; Pereira and Silva, 2022). Domain Services are employed to handle application tasks and technical code that is not part of the core business logic, while Entities and Value Objects represent different entities and their relationships in the domain model. These concepts contribute to modeling complex business domains and enabling object-oriented representations of domain concepts.

5.5. ACL for maintaining domain integrity

When dealing with different bounded contexts, interacting with external systems or refactoring legacy systems, the risk of corrupting a domain model with concepts and structures from another context is high. The ACL acts as a safeguard, ensuring that each context's domain remains untainted and independent. The concept of the ACL is discussed in some studies (Peng and Hu, 2007; Josélyne et al., 2018; Landre et al., 2006; Le et al., 2018). It is utilized in the studies to manage dependencies between different domains, preventing the contamination of contexts, safeguard the domain model of a Bounded Context from changes in other contexts, segregating application-specific elements from domain-specific components in microservices and maintaining consistency between aggregates. Studies also suggest that ACL shields new contexts from potential contaminations or disruptions originating in legacy contexts.

5.6. All hands on deck: It is not only about software engineers

Based on the data extracted from the literature, the implementation of DDD involves the active participation of various key stakeholders who play critical roles in ensuring its successful application. These stakeholders encompass a diverse set of roles, including Software Engineers, Architects, Project Managers, and Domain Experts. Software Engineers hold a central position in the DDD implementation process, responsible for translating the domain model into executable code, implementing domain-specific logic, and aligning the software architecture with domain requirements (Singjai et al., 2021b; Maddodi et al., 2020; Peng and Hu, 2007; Snoeck and Wautelet, 2022; Özkan et al., 2023). Architects contribute strategic guidance and oversight to the overall software design, collaborating with Software Engineers and other stakeholders to define high-level architecture, bounded contexts, and domain models that effectively capture domain knowledge (Maddodi et al., 2020; Kapferer and Zimmermann, 2021; Da Silva et al., 2022).

Project Managers play a vital role in coordinating DDD implementation efforts, ensuring alignment with project goals, resource allocation, and effective communication between teams and stakeholders (Peng and Hu, 2007; Uludağ et al., 2018). Domain Experts, possessing in-depth knowledge of the specific business domain, actively contribute

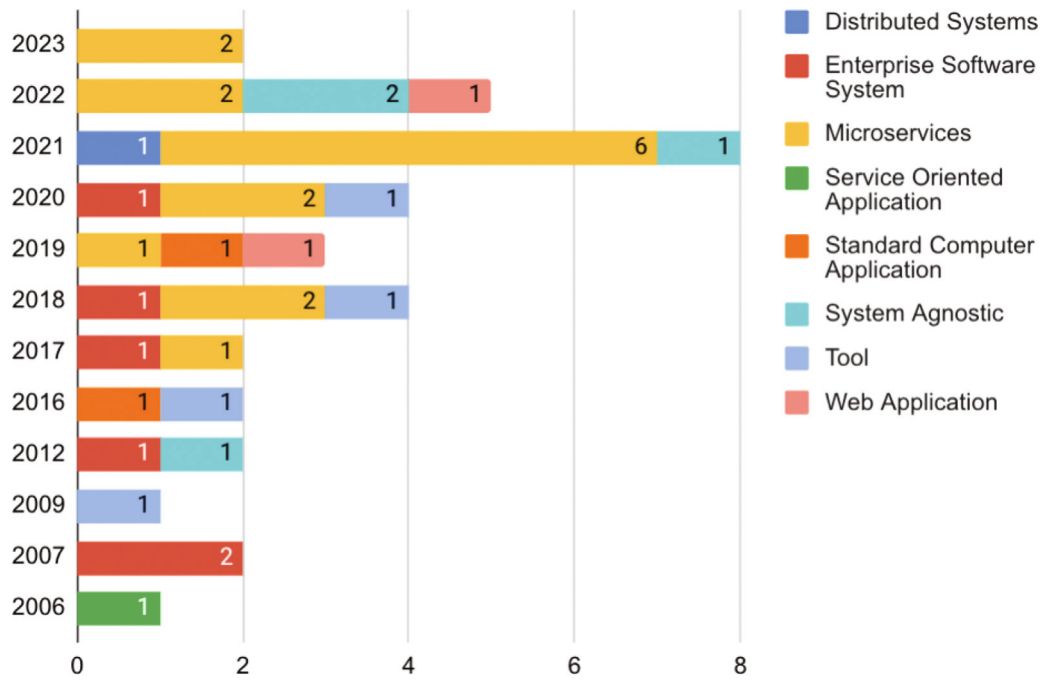


Fig. 6. DDD studies grouped by their software systems and publication years.

to refining the domain model, identifying key domain concepts, and validating the software's alignment with real-world domain requirements (Singjai et al., 2021b; Uludağ et al., 2018; Da Silva et al., 2022).

Moreover, the literature also identifies other relevant participants involved in the DDD implementation, including QA Engineers, DevOps Engineers, UX Designers, Business Analysts, Systems Analysts, Data Modeling Specialists, and end-users (Peng and Hu, 2007; Braun et al., 2021a; Özkan et al., 2023; Oukes et al., 2021; Häser et al., 2016; Bündler and Kuchen, 2019). The successful implementation of DDD requires close collaboration and communication among these diverse stakeholders to ensure that the developed software system effectively represents the domain's complexity and meets the business needs. The active engagement of software engineers, architects, project managers, and domain experts, along with the contributions from other relevant participants, enables the application of DDD principles to create robust and domain-centric software solutions. The involvement of these stakeholders enhances the alignment of software systems with real-world domain requirements and fosters the development of high-quality and effective software solutions.

Our review adds a nuanced view of how this collaboration evolves in practice. Rather than treating the developer–domain expert relationship as static, the primary studies reflect varied degrees of maturity in this interaction. For instance, some report structured practices such as Event Storming workshops (Uludağ et al., 2018), while others highlight onboarding challenges and communication gaps (Özkan et al., 2023; Wanderley and Da Silveria, 2012). This range of experiences suggests that bridging the gap is not a one-time activity, but an ongoing and often iterative process influenced by tooling, team experience, and organizational context — a perspective less emphasized in prior discussions.

5.7. Evaluation of DDD effectiveness

The evaluation of DDD effectiveness varies across studies. Some studies present empirical evaluations based on metrics, user feedback, or controlled experiments (Singjai et al., 2021b; Maddodi et al., 2020; Uludağ et al., 2018; Braun et al., 2021b; Danenas and Garsva, 2012;

Özkan et al., 2023; Mayer and Weinreich, 2018). However, several studies lack specific evaluation results or empirical evidence (Perillo et al., 2009; Krause et al., 2020; Ding et al., 2020; Wanderley and Da Silveria, 2012; Abeck et al., 2019; Bündler and Kuchen, 2019).

To establish a more comprehensive understanding of the benefits and potential challenges of implementing DDD in different software development scenarios, further empirical research is needed. Researchers and practitioners should focus on collecting concrete metrics, conducting real-world case studies, and measuring the perception of practitioners. We believe that it is also essential to involve human metrics, such as user feedback and assessments from domain experts, architects, and experienced developers. Considering the challenges in implementing DDD, human-centric evaluations can provide valuable insights into the effectiveness of DDD and its impact on the software development process.

5.8. Rise of the microservices in DDD

Our SLR reveals a significant shift in the popularity of microservices in DDD studies starting 2017 and at peak in 2021. As depicted in Fig. 6, prior to 2017, credible DDD studies specifically focusing on microservices were scarce, indicating a lack of attention in this area. However, from 2017 onwards, we observed a notable increase in the number of studies exploring the application of DDD principles in the context of microservices. This trend aligns with the findings of a Systematic Mapping Study on Microservices conducted by Hamzehlou et al. (2018) in 2018, which also reports a rising number of evaluation papers, suggesting that microservices have been gaining wider implementation in real-world software development scenarios.

Furthermore, Vigiato et al. (2018) in 2018 highlights the increasing popularity of microservices architectures, further emphasizing the growing relevance and adoption of microservices in the software industry. In a similar vein, Knoche and Hasselbring (2019) in 2019 notes that many companies are actively considering whether microservices are a viable option for their applications, indicating the considerable interest and attention surrounding the adoption of microservices-based architectures.

The convergence of our findings with those of the previously mentioned studies and experts further corroborates the growing significance

of microservices in the domain of software development, especially in the context of DDD methodologies. The upward trend in microservices adoption is evident from the year 2017 onwards, indicating a turning point in the landscape of software development practices. Beyond identifying the rise in microservices, our study offers novel insights into how DDD contributes to shaping modularity in such architectures. In particular, we observe that practitioners increasingly rely on DDD patterns like Bounded Contexts and Aggregates to define service boundaries and promote semantic clarity. While prior studies may suggest that microservices benefit from domain-aligned decomposition, our findings ground this assumption in a broader empirical base across 36 peer-reviewed studies. This synthesis provides stronger evidence for how DDD actively influences microservice design practices in both academic and industrial settings.

5.9. Challenges and considerations in implementing DDD effectively

To ensure the successful implementation of DDD, it is essential to employ DDD patterns accurately and efficiently. Proper utilization of DDD principles demands a skilled team consisting of experienced developers, domain experts, and seasoned architects (Özkan et al., 2023). The significance of experience in DDD cannot be overstated, as we have observed from various studies that onboarding to DDD is not a straightforward process, and the expertise of developers significantly influences the success of the implementation (Kapferer and Zimmermann, 2021; Özkan et al., 2023).

Implementing DDD effectively requires a substantial amount of attention and effort, particularly when integrating it into existing software systems. The transition from traditional software development approaches to DDD can be challenging, and the process demands careful planning, training, and a clear understanding of the DDD concepts and principles. Seasoned developers with prior experience in DDD are more likely to grasp the complexities and intricacies of the approach, resulting in better-designed and more maintainable software systems (Hippchen et al., 2017). On the other hand, inexperienced developers may struggle to fully utilize the benefits of DDD, leading to potential issues in the implementation process.

Furthermore, the involvement of domain experts plays a crucial role in the success of DDD implementation. Their deep understanding of the business domain is invaluable in guiding the development team to create accurate and effective domain models (Vural and Koyuncu, 2021). However, acquiring experienced domain experts can be challenging, especially in specialized domains. Their scarcity may pose hurdles in the adoption of DDD, but their involvement is crucial for crafting domain models that truly reflect the intricacies and requirements of the real-world domain.

5.10. Academia-industry collaboration

The academic studies in our SLR predominantly emphasize the theoretical aspects of DDD. They explore the foundational principles and patterns of DDD, such as Ubiquitous Language, Bounded Context, Entities, and Value Objects. These studies often propose novel frameworks, tools and methodologies that extend the DDD concepts to specific problem domains. While they provide valuable insights into the theoretical underpinnings of DDD, they might lack empirical validation and practical implementation in real-world projects.

On the other hand, we experienced that the industry-focused studies put DDD principles into practice in various software systems. They showcase how DDD is applied to real-world projects and explore its impact on system design, architecture, and development. These studies often involve case studies and experience reports from software engineers, architects, and domain experts. The emphasis is on practical implementation, challenges faced, and lessons learned while applying DDD in complex software projects.

Our findings indicate a healthy mix of academic and industry studies, reflecting the growing interest in DDD from both research and practical application perspectives. While academic studies contribute theoretical advancements and innovative approaches, industry-focused studies demonstrate the real-world applicability and effectiveness of DDD principles in solving complex software design and development challenges. More interaction between academia and industry in DDD research could potentially facilitate the transfer of knowledge and best practices, contributing to the further advancement and adoption of DDD principles in real-world software development scenarios, as successfully demonstrated in Özkan et al. (2023).

5.10.1. Contrasting academic and industry perspectives

While our study includes a balanced set of academic (58.3%) and industry (36.1%) studies — with a small number of hybrid cases (5.6%) — the practical perspectives from industry deserve further distinction. Academic research on DDD tends to focus on conceptual modeling, tool development (e.g., DSLs, context mapping tools), and theoretical discussions on patterns such as Bounded Contexts or Entities. These studies often propose frameworks or methodologies but may lack concrete empirical validation. In contrast, industry studies are generally more experience-driven, describing real-world applications of DDD in large-scale systems such as microservices, enterprise refactoring, or system modernization. These studies prioritize architectural pragmatism, performance concerns, and integration challenges, and often highlight the social or organizational implications of DDD adoption (e.g., communication, onboarding, and team alignment).

For example, the academic studies often examine how to formalize context boundaries using modeling tools like Context Mapper or domain-specific languages, whereas the industry studies report practical experience applying bounded contexts in cross-functional teams or resolving communication issues between developers and domain experts. Moreover, industry papers tend to use anecdotal or case-based evidence, while academic studies more frequently advocate for abstract, generalizable models. This contrast reveals complementary strengths: academia offers methodological structure and design guidance, while industry grounds these concepts in complex, evolving development environments. Recognizing this distinction helps to contextualize the relevance and maturity of DDD research across settings.

5.11. A critical analysis of research quality in the DDD literature

The analyzed studies demonstrate varying degrees of adherence to DDD principles and the effective application of DDD concepts. While some studies effectively apply DDD principles, others exhibit shortcomings in implementation and evaluation. To enhance research quality, future studies should emphasize the consistent use of Ubiquitous Language, explicit definition of Bounded Contexts, proper utilization of Domain Events and DDD patterns, and robust evaluation methodologies. Additionally, studies should openly discuss the challenges and limitations of DDD, contributing to a more comprehensive understanding of its potential and practical implications in software development.

Several studies employed the notion of Ubiquitous Language, which facilitates effective communication between stakeholders and software designers by creating a shared vocabulary. However, a few studies failed to implement this language consistently, potentially hindering communication and understanding among team members. Future research should emphasize the importance of adopting and consistently applying Ubiquitous Language throughout the development process to ensure better alignment and clarity.

Bounded Context, another key concept in DDD, helps define explicit boundaries where specific domain models apply (Evans, 2004; Vernon, 2016). While some studies effectively utilized Bounded Context to organize and segregate domains, others did not explicitly mention or implement this principle. Ensuring clear identification and definition of Bounded Contexts is crucial for managing interactions between

different models and contexts, and future studies should focus on this aspect to enhance research quality.

The application of Domain Events to trigger behavior across multiple aggregates within a Bounded Context was observed in some studies. However, not all studies effectively leveraged this concept. Implementing Domain Events can improve event-driven communication and coordination between domain components, promoting a more cohesive and flexible architecture (Evans, 2004; Vernon, 2016). Future research should explore the potential benefits of Domain Events in event-driven systems and provide concrete examples of their implementation.

Several studies showcased the use of Aggregates, Entities, and Value Objects to model complex business domains. While these concepts were well-implemented in some cases, others lacked specific details or provided inadequate explanations about their usage. To bolster research quality, studies should strive for comprehensive and consistent implementation of these DDD patterns, ensuring a clear understanding of their roles and relationships within the domain model.

We observed that Strategic Design patterns such as Core Domain, Shared Kernel, and upstream–downstream relationships were underreported in most studies. This suggests a possible gap in the literature, where tactical DDD patterns are emphasized, but strategic alignment is less frequently explored or documented. Future research should pay more attention to these higher-level design patterns to understand their role in system architecture and team collaboration.

One significant aspect that emerged from the data was the lack of specific evaluation methods in some studies. While some research incorporated empirical evaluations, others relied solely on proposing methodologies or demonstrating case studies without conducting rigorous evaluations. Future studies should adopt more robust evaluation methods, such as controlled experiments, surveys, or user feedback, to assess the effectiveness of DDD implementation and provide empirical evidence to support their findings.

Moreover, most studies did not discuss the challenges and limitations associated with DDD adoption, leaving potential gaps in addressing concerns or drawbacks of the approach. To improve research quality, studies should acknowledge and discuss both the advantages and limitations of DDD, offering a balanced view to readers and practitioners. This will help in understanding the practical implications of DDD and guide future research and implementation efforts.

Future research can provide a more comprehensive and balanced analysis of DDD by addressing these areas to contribute to the advancement of knowledge and practice in the field of software development.

6. Implications

In this section, we introduce the implications of our findings for both practitioners and researchers. We draw on the results presented in our RQs to ground our implications in the evidence synthesized from the primary studies.

Implications for Practitioners. One of the primary implications for practitioners is the recognition of the diverse benefits that DDD can bring to various software systems. The application of DDD principles has been shown to improve the modularity, maintainability, and scalability of microservices-based applications (Section 4.3), thereby enabling organizations to handle complex and distributed systems more effectively (Singjai et al., 2021b; Josélyne et al., 2018; Wang et al., 2022). Practitioners can leverage these insights to enhance their software architecture and development practices by adopting DDD patterns such as Bounded Contexts and Aggregates to define clear service boundaries and promote decoupling.

Moreover, the adoption of DDD in enterprise software systems has proven valuable in addressing challenges related to legacy system modernization and increasing architectural complexity (Maddodi et al., 2020; Peng and Hu, 2007; Uludağ et al., 2018). Section 4.2 illustrates how DDD facilitates flexible and maintainable architectures by aligning

software components with evolving business needs. Practitioners aiming to modernize legacy systems should consider using context maps and ACLs (as highlighted in Section 4.5) to incrementally isolate and refactor legacy code while preserving domain integrity.

Additionally, DDD has demonstrated benefits in improving code quality and software construction in web applications and development tools (Le et al., 2022; Perillo et al., 2009; Kapferer and Zimmermann, 2020). In Section 4.3, studies report that domain annotations and modeling tools help practitioners maintain separation of concerns and automate parts of the development process. These insights can guide practitioners in selecting DDD-aligned tools that improve code maintainability (Le et al., 2022) and streamline software generation.

However, DDD is not without its challenges. As discussed in Section 4.5, one of the most significant obstacles is the steep learning curve, especially for developers new to DDD (Özkan et al., 2023; Hipchen et al., 2017). Misunderstandings in the application of patterns, unclear decomposition strategies, and the misalignment between teams and domains are common pitfalls. To mitigate this, practitioners should invest in onboarding initiatives, including dedicated DDD training sessions, domain modeling workshops (e.g., Event Storming (Uludağ et al., 2018)), and mentorship from experienced architects (see Table 8).

Implications for Researchers. Despite the progress made in understanding the application and benefits of DDD, Section 4.6 shows that only a limited number of studies (approximately 39%) employ empirical evaluations. Future research should prioritize the design of empirical studies with concrete evaluation metrics — such as team productivity, domain alignment, and architectural complexity — to measure the actual impact of DDD adoption in industrial settings (Singjai et al., 2021b; Maddodi et al., 2020; Uludağ et al., 2018).

Furthermore, Section 4.5 indicates that the complexity of managing domain boundaries, integrating DDD with legacy systems, and supporting DSL-based tooling are persistent technical challenges. Researchers should explore novel techniques — e.g., automated decomposition, model-code synchronization, and refactoring strategies — to support scalable DDD adoption in complex environments (Braun et al., 2021b; Özkan et al., 2023).

Another important research direction involves studying stakeholder dynamics. Section 4.7 emphasizes the critical role of collaboration between software engineers, architects, and domain experts in the success of DDD. Yet, communication breakdowns and a lack of domain understanding remain major barriers (Wanderley and Da Silveira, 2012; Da Silva et al., 2022). Researchers could investigate socio-technical methods (e.g., participatory modeling (Voinov and Bousquet, 2010; Brandolini, 2019), collaborative tooling (Famelis et al., 2012)) to bridge this gap and formalize co-design practices.

In addition, Section 4.6 reveals that studies vary widely in how they evaluate DDD. Developing standardized frameworks and reproducible evaluation methodologies will help unify the field and allow comparison across contexts (Özkan et al., 2023; Le et al., 2022). Incorporating both technical metrics (e.g., coupling, cohesion) and human-centered metrics (e.g., developer perception, onboarding effort) can lead to a more holistic understanding of DDD's effectiveness.

Based on our findings, future research should prioritize empirical studies that not only report qualitative insights but also collect large-scale (e.g. via repository mining for DDD similar to Babur et al. (2024)), longitudinal and team-level data. While several studies use focus groups and interviews, we observed a lack of sustained evaluations on aspects such as maintainability, modularity drift over time, or onboarding effectiveness. Studies like Braun et al. (2021b,a) and Özkan et al. (2023) demonstrate the potential of combining qualitative feedback with structured metrics, yet these efforts remain isolated. To build on this, researchers could design action research or longitudinal case studies that track DDD adoption over multiple sprints or releases, focusing on how domain alignment evolves, how design debt is managed, and how development teams adapt. Similarly, design-science-based evaluations

of modeling tools (e.g., Context Mapper) can shed light on how tooling impacts the domain-code connection in practical workflows. These directions would help close both methodological and tooling-related gaps identified in current literature.

By addressing these research gaps, scholars can contribute not only to the theoretical advancement of DDD but also to practical guidelines, tool support, and empirical validation frameworks that help teams successfully adopt DDD in complex software projects.

7. Limitations and threats to validity

Some limitations and validity considerations are applicable for SLR studies (Petersen et al., 2015, 2008). The limitations and threats to the validity of this study are discussed in this section.

Timeframe and Currency. One limitation is the timeframe of the literature search and the currency of the included studies. The SLR is based on data available up to June 2023, and relevant studies published after that date might have been missed. The field of DDD is rapidly evolving, and newer developments or trends may not be fully captured in this review.

Quality of Studies. The quality of individual studies can impact the overall reliability of the SLR findings. The included studies vary in terms of methodological rigor, sample size, and research design. Some studies might have limitations or biases that could influence their results and subsequent conclusions. Nonetheless, we endeavored to rigorously exclude studies of low quality by systematically applying inclusion and exclusion criteria as well as conducting thorough quality assessments.

Inclusion and Exclusion Criteria. The effectiveness of an SLR largely depends on the clarity and appropriateness of the inclusion and exclusion criteria. Despite our efforts to define comprehensive criteria, there is a possibility of unintentionally excluding relevant studies or including studies that do not precisely fit the research questions. To mitigate this potential limitation, a rigorous approach was adopted. Two authors independently applied predefined inclusion and exclusion criteria to a randomly selected pool of 60 studies. The level of agreement between the two authors was assessed using Cohen's Kappa coefficient (Cohen, 1960), a widely accepted measure of inter-rater reliability. Encouragingly, the analysis revealed a perfect agreement between the authors in their decisions regarding study inclusion and exclusion within the randomly sampled studies. This robust agreement underscores the reliability of the selection process and enhances the credibility of the findings.

Bias in Data Extraction. A limitation related to the validity of data extraction in this study is the potential for human error or bias during the data extraction process. Despite efforts to maintain consistency and accuracy, the interpretation of information from primary studies could be influenced by the subjectivity of the researchers. Different individuals extracting data may interpret the findings differently or unintentionally introduce errors, leading to inconsistencies in the extracted data. To address this limitation, a careful and systematic approach was employed, and multiple researchers were involved in the data extraction process. Inter-rater reliability checks and discussions among the researchers were conducted to ensure agreement and minimize discrepancies. However, it is essential to acknowledge that the possibility of human-related errors cannot be entirely eliminated, and they may have some impact on the final results and conclusions of the systematic literature review.

Publication Bias. One potential threat to validity is publication bias. The studies included in this SLR were obtained from various academic databases and sources, which might not represent the entire body of literature on DDD. Some studies might not have been published or accessible, leading to a skewed representation of the research landscape. Nevertheless, to mitigate this bias, we performed a comprehensive search across multiple academic databases, including Google Scholar and Scopus during the snowballing process. This extensive

search approach was aimed at encompassing a wide range of academic sources and reducing potential biases in the selection of studies.

Language Bias. The SLR focused primarily on studies published in English, which might introduce a language bias. Relevant research conducted in other languages could have been omitted, potentially impacting the comprehensiveness of the review.

Agreement Level During Quality Assessment. Another potential threat to validity is the lack of assessment of agreement levels between assessors during the quality assessment stage. While Cohen's Kappa was used to evaluate inter-rater reliability during the inclusion/exclusion stage, no similar measure was applied during the quality assessment of the included studies. This omission could introduce subjectivity and potential bias in the quality assessment process, as individual assessors may have differing interpretations of quality criteria. However, during the quality assessment stage, in cases of disagreement or doubt, the assessors consulted with each other to reach a consensus. This approach used to ensure that the quality assessments were rigorous, minimized potential biases, and allowed discrepancies to be resolved systematically.

Study-Specific Validity Considerations. In addition to general methodological considerations, we acknowledge several threats specific to our study. First, our focus on implementation-related search terms (e.g., *refactor**, *improv**, *optimiz**) may have led to the exclusion of relevant DDD papers that are more theoretical or descriptive in nature. This trade-off was a deliberate decision to ensure practical relevance, but it may affect the completeness of our study population. Second, while our inclusion of forward snowballing from Evans' seminal book aimed to enrich the dataset with practically grounded work, it may also have introduced a bias toward studies that cite this particular source, possibly underrepresenting alternative DDD schools of thought. Finally, our coding and classification of DDD patterns — especially in distinguishing strategic vs. tactical applications — required interpretation and contextual judgment. Despite using a predefined form and multiple authors reviewing the coding, subjectivity may still have influenced the final categorization.

8. Conclusion

This SLR provides a comprehensive overview of the effectiveness, challenges, and implications of adopting DDD in software development. Through the analysis of a diverse set of primary studies, we identified key stakeholders involved in the implementation of DDD, including Software Engineers, Architects, Project Managers, and Domain Experts. These stakeholders play critical roles in ensuring the successful application of DDD principles and practices in various software systems.

The findings highlight the significance of Ubiquitous Language as a core principle of DDD, emphasizing its role in facilitating effective communication and collaboration among team members. Bounded Contexts and the decomposition of microservices stand out as fundamental concepts in DDD, enabling the creation of scalable and maintainable software systems. Domain Services, Entities, and Value Objects serve as tactical DDD patterns, contributing to the modeling of complex business domains.

The implementation of DDD demands considerable attention and effort, particularly in transitioning from traditional software development approaches to DDD. Experienced developers and domain experts play crucial roles in effectively implementing DDD concepts. The involvement of domain experts is especially critical for crafting domain models that accurately represent real-world domain requirements.

The SLR also highlights the rise of microservices in the context of DDD, indicating a significant shift in the popularity of microservices-based architectures since 2017. However, further empirical research is needed to comprehensively understand the benefits and challenges of implementing DDD in different software development scenarios.

The collaboration between academia and industry in DDD research is evident, with academic studies contributing theoretical advancements and innovative approaches, while industry-focused studies showcase practical implementation and real-world applicability. This collaboration can facilitate knowledge transfer and best practices, contributing to the advancement and adoption of DDD principles in software development.

A critical analysis of research quality in the DDD literature reveals varying degrees of adherence to DDD principles and the effective application of DDD concepts. Future studies should emphasize the consistent use of Ubiquitous Language, clear definition of Bounded Contexts, proper utilization of Domain Events and DDD patterns, and robust evaluation methodologies. Moreover, studies should openly discuss the challenges and limitations of DDD to provide a comprehensive understanding of its potential and practical implications.

DDD offers valuable insights and practices for software development, fostering communication, and collaboration among stakeholders while modeling complex business domains. However, its successful implementation depends on the expertise of stakeholders, proper utilization of DDD principles, and the adoption of robust evaluation methods. As software systems continue to grow in complexity, DDD remains a relevant and promising approach for developing domain-centric and scalable software solutions.

CRedit authorship contribution statement

Ozan Özkan: Writing – review & editing, Writing – original draft, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Önder Babur:** Validation, Supervision. **Mark van den Brand:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Abeck, S., Schneider, M., Quirmbach, J.-P., Klarl, H., Urbaczek, C., Zogaj, S., 2019. A context map as the basis for a microservice architecture for the connected car domain. <http://dx.doi.org/10.18420/INF2019.18>.
- Avram, A., Marinescu, F., 2006. Domain-driven design quickly: a summary of eric evans' domain-driven design. In: *Enterprise Software Development Series*, C4Media.
- Babur, Ö., Constantinou, E., Serebrenik, A., 2024. Language usage analysis for EMF metamodels on GitHub. *Empir. Softw. Eng.* 29 (1), 23.
- Bertoni, A., 2020. Data-driven design in concept development: systematic review and missed opportunities. *Proc. Des. Soc.: Design Conf.* 1, 101–110. <http://dx.doi.org/10.1017/dsd.2020.4>.
- Brandolini, A., 2013. Introducing EventStorming – an explosive modeling technique for lean discovery. <https://ziobrando.blogspot.com/2013/11/introducing-eventstorming.html>. (Accessed April 2025).
- Brandolini, A., 2019. Introducing EventStorming: An Act of Deliberate Collective Learning. *Leapub*.
- Braun, S., Bieniussa, A., Elberzhager, F., 2021a. Advanced domain-driven design for consistency in distributed data-intensive systems. In: *Proceedings of the 8th Workshop on Principles and Practice of Consistency for Distributed Data*. ACM, Online United Kingdom, pp. 1–12. <http://dx.doi.org/10.1145/3447865.3457969>.
- Braun, S., Deßloch, S., Wolff, E., Elberzhager, F., Jedlitschka, A., 2021b. Tackling consistency-related design challenges of distributed data-intensive systems: An action research study. In: *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement*. ESEM, ACM, Bari Italy, pp. 1–11. <http://dx.doi.org/10.1145/3475716.3475771>.
- Bünder, H., Kuchen, H., 2019. A model-driven approach for behavior-driven GUI testing. In: *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*. ACM, Limassol Cyprus, pp. 1742–1751. <http://dx.doi.org/10.1145/3297280.3297450>.
- Buschmann, F. (Ed.), 1996. *Pattern-Oriented Software Architecture: A System of Patterns*. Wiley, Chichester ; New York.
- Camilli, M., Colarusso, C., Russo, B., Zimeo, E., 2023. Actor-driven decomposition of microservices through multi-level scalability assessment. *ACM Trans. Softw. Eng. Methodol.* 3583563. <http://dx.doi.org/10.1145/3583563>.
- Cohen, J., 1960. A coefficient of agreement for nominal scales. *Educ. Psychol. Meas.* 20 (1), 37–46. <http://dx.doi.org/10.1177/001316446002000104>.
- Da Silva, C.E., Gomes, E.L., Basu, S.S., 2022. BPM2DDD: A systematic process for identifying domains from business processes models. *Software* 1 (4), 417–449. <http://dx.doi.org/10.3390/software1040018>.
- Danenas, P., Garsva, G., 2012. Domain driven development and feature driven development for development of decision support systems. In: *Skersys, T., Butleris, R., Butkiene, R. (Eds.), Information and Software Technologies*. vol. 319, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 187–198. http://dx.doi.org/10.1007/978-3-642-33308-8_16.
- De Goey, H., Hilletoft, P., Eriksson, L., 2019. Design-driven innovation: A systematic literature review. *Eur. Bus. Rev.* 31 (1), 92–114. <http://dx.doi.org/10.1108/EBR-09-2017-0160>.
- Ding, Y., Wang, L., Li, S., Wang, X., Zhang, J., 2020. Enterprise service application architecture based on domain driven model design. In: *2020 2nd International Conference on Information Technology and Computer Application*. ITCA, IEEE, Guangzhou, China, pp. 778–784. <http://dx.doi.org/10.1109/ITCA52113.2020.00167>.
- Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., Safina, L., 2017. Microservices: Yesterday, today, and tomorrow. <http://dx.doi.org/10.48550/arXiv.1606.04036>, arXiv:1606.04036.
- Evans, E., 2004. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, Boston.
- Famelis, M., Salay, R., Chechik, M., 2012. Partial models: Towards modeling and reasoning with uncertainty. In: *Proceedings of the 34th International Conference on Software Engineering*. ICSE, IEEE, pp. 573–583.
- Fritzsch, J., Bogner, J., Wagner, S., Zimmermann, A., 2019. Microservices migration in industry: intentions, strategies, and challenges. In: *2019 IEEE International Conference on Software Maintenance and Evolution*. ICSME, IEEE, Cleveland, OH, USA, pp. 481–490. <http://dx.doi.org/10.1109/ICSME.2019.00081>.
- Gamma, E., Helm, R., Johnson, R.E., Vlissides, J., 2009. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, OCLC: 624525693.
- Giray, G., Bennis, K.E., Köksal, Ö., Babur, Ö., Tekinerdogan, B., 2023. On the use of deep learning in software defect prediction. *J. Syst. Softw.* 195, 111537. <http://dx.doi.org/10.1016/j.jss.2022.111537>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0164121222002138>.
- Hammarström, P., Herzog, E., 2016. Experience from integrating domain driven software system design into a systems engineering organization. *INCOSE Int. Symp.* 26 (1), 1192–1203. <http://dx.doi.org/10.1002/j.2334-5837.2016.00220.x>.
- Hamzehlou, M.S., Sahibuddin, S., Salah, K., 2018. A systematic mapping study on microservices. In: *Advances in Intelligent Systems and Computing*. Springer International Publishing, pp. 1079–1090. http://dx.doi.org/10.1007/978-3-319-99007-1_100.
- Häser, F., Felderer, M., Breu, R., 2016. Is business domain language support beneficial for creating test case specifications: A controlled experiment. *Inf. Softw. Technol.* 79, 52–62. <http://dx.doi.org/10.1016/j.infsof.2016.07.001>.
- Hippchen, B., Giessler, P., Steinegger, R.H., Schneider, M., Abeck, S., 2017. Designing microservice-based applications by using a domain-driven design approach.
- Joselyne, M.I., Bajpai, G., Nzanywayingoma, F., 2021. A systematic framework of application modernization to microservice based architecture. In: *2021 International Conference on Engineering and Emerging Technologies*. ICEET, IEEE, Istanbul, Turkey, pp. 1–6. <http://dx.doi.org/10.1109/ICEET53442.2021.9659783>.
- Josélyne, M.I., Tuheirwe-Mukasa, D., Kanagwa, B., Balikuddembe, J., 2018. Partitioning microservices: A domain engineering approach. In: *Proceedings of the 2018 International Conference on Software Engineering in Africa*. ACM, Gothenburg Sweden, pp. 43–49. <http://dx.doi.org/10.1145/3195528.3195535>.
- Kapferer, S., Zimmermann, O., 2020. Domain-driven service design. In: *Service-Oriented Computing*. In: *Communications in Computer and Information Science*, Springer International Publishing, Cham, pp. 189–208.
- Kapferer, S., Zimmermann, O., 2021. Domain-driven architecture modeling and rapid prototyping with context mapper. In: *Hammoudi, S., Pires, L.F., Selić, B. (Eds.), Model-Driven Engineering and Software Development*. vol. 1361, Springer International Publishing, Cham, pp. 250–272. http://dx.doi.org/10.1007/978-3-030-67445-8_11.
- Kitchenham, B., Charters, S., 2007. Guidelines for performing systematic literature reviews in software engineering 2.
- Knoche, H., Hasselbring, W., 2019. Drivers and barriers for microservice adoption – A Survey among professionals in Germany. *Enterp. Model. Inf. Syst. Archit. (EMISAJ)* 1:1–35. <http://dx.doi.org/10.18417/EMISA.14.1>.
- Koryl, M., 2017. Active resources concept of computation for enterprise software. *Arch. Control. Sci.* 27 (2), 279–291. <http://dx.doi.org/10.1515/acsc-2017-0018>.
- Krause, A., Zirkelbach, C., Hasselbring, W., Lenga, S., Kroger, D., 2020. Microservice decomposition via static and dynamic analysis of the monolith. In: *2020 IEEE International Conference on Software Architecture Companion*. ICSC-C, IEEE, Salvador, Brazil, pp. 9–16. <http://dx.doi.org/10.1109/ICSC-C50368.2020.00011>.

- Kuhrmann, M., Fernández, D.M., Daneva, M., 2017. On the pragmatic design of literature studies in software engineering: An experience-based guideline. *Empir. Softw. Eng.* 22 (6), 2852–2891. <http://dx.doi.org/10.1007/s10664-016-9492-y>.
- Landre, E., Wesenberg, H., Olmheim, J., 2007. Agile enterprise software development using domain-driven design and test first. In: Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion. ACM, Montreal Quebec Canada, pp. 983–993. <http://dx.doi.org/10.1145/1297846.1297967>.
- Landre, E., Wesenberg, H., Rønneberg, H., 2006. Architectural improvement by use of strategic level domain-driven design. In: Companion To the 21st ACM SIGPLAN Symposium on Object-Oriented Programming Systems, Languages, and Applications. ACM, Portland Oregon USA, pp. 809–814. <http://dx.doi.org/10.1145/1176617.1176728>.
- Le, D.M., Dang, D.-H., Nguyen, V.-H., 2016. Domain-driven design patterns: A metadata-based approach. In: 2016 IEEE RIVF International Conference on Computing & Communication Technologies, Research, Innovation, and Vision for the Future. RIVF, IEEE, Hanoi, pp. 247–252. <http://dx.doi.org/10.1109/RIVF.2016.7800302>.
- Le, D.M., Dang, D.-H., Nguyen, V.-H., 2018. On domain driven design using annotation-based domain specific language. *Comput. Lang. Syst. Struct.* 54, 199–235. <http://dx.doi.org/10.1016/j.cl.2018.05.001>.
- Le, D.M., Dang, D.-H., Vu, H.T., 2019. JDomainApp: A module-based domain-driven software framework. In: Proceedings of the Tenth International Symposium on Information and Communication Technology - SoICT 2019. ACM Press, Hanoi, Ha Long Bay, Viet Nam, pp. 399–406. <http://dx.doi.org/10.1145/3368926.3369657>.
- Le, D.M., Nguyen, A.P., Tran, L.Q., Le, H.T., Tong, H.V.-A., 2022. Generating multi-platform single page applications: A hierarchical domain-driven design approach. In: The 11th International Symposium on Information and Communication Technology. ACM, Hanoi Vietnam, pp. 344–351. <http://dx.doi.org/10.1145/3568562.3568566>.
- Lewis, J., Fowler, M., 2014. *Microservices: A definition of this new architectural term*. MartinFowler.com 25 (14–26), 12.
- Lübke, D., Zimmermann, O., Pautasso, C., Zdun, U., Stocker, M., 2019. Interface evolution patterns: Balancing compatibility and extensibility across service life cycles. In: Proceedings of the 24th European Conference on Pattern Languages of Programs. In: EuroPlop '19, Association for Computing Machinery, New York, NY, USA, <http://dx.doi.org/10.1145/3361149.3361164>.
- Maddodi, G., Jansen, S., Overeem, M., 2020. Aggregate architecture simulation in event-sourcing applications using layered queuing networks. In: Proceedings of the ACM/SPEC International Conference on Performance Engineering. ACM, Edmonton AB Canada, pp. 238–245. <http://dx.doi.org/10.1145/3358960.3375797>.
- Mayer, B., Weinreich, R., 2018. An approach to extract the architecture of microservice-based software systems. In: 2018 IEEE Symposium on Service-Oriented System Engineering. SOSE, IEEE, Bamberg, pp. 21–30. <http://dx.doi.org/10.1109/SOSE.2018.00012>.
- Nilsson, J., 2006. *Applying Domain-Driven Design and Patterns: With Examples in C# and NET*. Addison-Wesley, Upper Saddle River, NJ.
- Ochoa, L., Hammad, M., Giray, G., Babur, Ö., Bennis, K., 2025. Characterising harmful API uses and repair techniques: Insights from a systematic review. *Comput. Sci. Rev.* 57, 100732.
- Oukes, P., Andel, M.V., Folmer, E., Bennett, R., Lemmen, C., 2021. Domain-driven design applied to land administration system development: Lessons from the Netherlands. *Land Use Policy* 104, 105379. <http://dx.doi.org/10.1016/j.landusepol.2021.105379>.
- Özkan, O., Babur, Ö., Van Den Brand, M., 2023. Refactoring with domain-driven design in an industrial context: an action research report. *Empir. Softw. Eng.* 28 (4), 94. <http://dx.doi.org/10.1007/s10664-023-10310-1>.
- Peng, S., Hu, Y., 2007. Ianticorruption: A domain-driven design approach to more robust integration. In: Companion To the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion. ACM, Montreal Quebec Canada, pp. 976–982. <http://dx.doi.org/10.1145/1297846.1297966>.
- Pereira, P., Silva, A.R., 2022. Towards transactional causal consistent microservices business logic. [arXiv:2212.11658](https://arxiv.org/abs/2212.11658).
- Perillo, J.R.C., Guerra, E.M., Fernandes, C.T., 2009. Daileon: A tool for enabling domain annotations. In: Proceedings of the Workshop on AOP and Meta-Data for Software Evolution. ACM, Genova Italy, pp. 1–4. <http://dx.doi.org/10.1145/1562860.1562867>.
- Petersen, K., Feldt, R., Mujtaba, S., Mattsson, M., 2008. Systematic mapping studies in software engineering. In: 12th International Conference on Evaluation and Assessment in Software Engineering. EASE, <http://dx.doi.org/10.14236/ewic/EASE2008.8>.
- Petersen, K., Vakkalanka, S., Kuzniar, L., 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. *Inf. Softw. Technol.* 64, 1–18. <http://dx.doi.org/10.1016/j.infsof.2015.03.007>.
- Popay, J., Roberts, H., Sowden, A., Petticrew, M., Arai, L., Rodgers, M., Britten, N., Roen, K., Duffy, S., 2006. Guidance on the Conduct of Narrative Synthesis in Systematic Reviews: A Product from the ESRC Methods Programme. Lancaster University, <http://dx.doi.org/10.13140/2.1.1018.4643>.
- Singjai, A., Zdun, U., Zimmermann, O., 2021a. Practitioner views on the interrelation of microservice APIs and domain-driven design: a grey literature study based on grounded theory. In: 2021 IEEE 18th International Conference on Software Architecture. ICSA, IEEE, Stuttgart, Germany, pp. 25–35. <http://dx.doi.org/10.1109/ICSA51549.2021.00011>.
- Singjai, A., Zdun, U., Zimmermann, O., Pautasso, C., 2021b. Patterns on deriving APIs and their endpoints from domain models. In: 26th European Conference on Pattern Languages of Programs. ACM, Graz Austria, pp. 1–15. <http://dx.doi.org/10.1145/3489449.3489976>.
- Snoeck, M., Wautelet, Y., 2022. Agile MERODE: A model-driven software engineering method for user-centric and value-based development. *Softw. Syst. Model.* 21 (4), 1469–1494. <http://dx.doi.org/10.1007/s10270-022-01015-y>.
- Stol, K.-J., Fitzgerald, B., 2018. The ABC of software engineering research. *ACM Trans. Softw. Eng. Methodol.* (TOSEM) 27 (3), 1–51.
- Torres, W., Van Den Brand, M.G.J., Serebrenik, A., 2021. A systematic literature review of cross-domain model consistency checking by model management tools. *Softw. Syst. Model.* 20 (3), 897–916. <http://dx.doi.org/10.1007/s10270-020-00834-1>.
- Uludağ, Ö., Haider, M., Kleehaus, M., Schimpfle, C., Matthes, F., 2018. Supporting large-scale agile development with domain-driven design. In: Garbajosa, J., Wang, X., Aguiar, A. (Eds.), *Agile Processes in Software Engineering and Extreme Programming*. vol. 314, Springer International Publishing, Cham, pp. 232–247. http://dx.doi.org/10.1007/978-3-319-91602-6_16.
- Vernon, V., 2013. *Implementing Domain-Driven Design*. Addison-Wesley, Upper Saddle River, NJ.
- Vernon, V., 2016. *Domain-Driven Design Distilled*. Addison-Wesley, Boston.
- Vigliato, M., Terra, R., Rocha, H., Valente, M.T., Figueiredo, E., 2018. Microservices in practice: A survey study. <http://dx.doi.org/10.48550/ARXIV.1808.04836>.
- Voinov, A., Bousquet, F., 2010. Modeling with stakeholders. *Environ. Model. Softw.* 25 (11), 1268–1281.
- Vural, H., Koyuncu, M., 2021. Does domain-driven design lead to finding the optimal modularity of a microservice? *IEEE Access* 9, 32721–32733. <http://dx.doi.org/10.1109/ACCESS.2021.3060895>.
- Wanderley, F., Da Silveria, D.S., 2012. A framework to diminish the gap between the business specialist and the software designer. In: 2012 Eighth International Conference on the Quality of Information and Communications Technology. IEEE, Lisbon, TBD, Portugal, pp. 199–204. <http://dx.doi.org/10.1109/QUATIC.2012.9>.
- Wang, S., Huang, L., Gao, A., Ge, J., Zhang, T., Feng, H., Satyarth, I., Li, M., Zhang, H., Ng, V., 2023. Machine/deep learning for software engineering: A systematic literature review. *IEEE Trans. Softw. Eng.* 49 (3), 1188–1231. <http://dx.doi.org/10.1109/TSE.2022.3173346>.
- Wang, Y., Li, S., Liu, H., Zhang, H., Pan, B., 2022. A reference architecture for blockchain-based traceability systems using domain-driven design and microservices. In: 2022 29th Asia-Pacific Software Engineering Conference. APSEC, IEEE, Japan, pp. 269–278. <http://dx.doi.org/10.1109/APSEC57359.2022.00039>.
- Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering. ACM, London England United Kingdom, pp. 1–10. <http://dx.doi.org/10.1145/2601248.2601268>.
- Zhao, J., Zhao, K., 2021. Applying microservice refactoring to object-oriented legacy system. In: 2021 8th International Conference on Dependable Systems and their Applications. DSA, IEEE, Yinchuan, China, pp. 467–473. <http://dx.doi.org/10.1109/DSA52907.2021.00070>.